



**Strathmore University**

*@iLabAfrica Centre*

---

# Applied Machine Learning

From First Principles to Production Systems

---

**Javier Serrano**  
**Emmanuel Lee Bundi**

Master in Data Science and Analytics  
DSA 8401

Nairobi, 2026

**Applied Machine Learning: From First Principles to Production Systems in the African Fintech Ecosystem**

Javier Serrano & Emmanuel Lee Bundi  
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,  
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.  
Draft edition — for enrolled students. Not for redistribution.

Typeset with L<sup>A</sup>T<sub>E</sub>X using the book class.  
Figures produced with TikZ; code listings with listings.

Contact: [jgarcia@strathmore.edu](mailto:jgarcia@strathmore.edu)

Version 0.1 — August 24, 2026

# Contents

---

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Preface</b>                                               | <b>ix</b> |
| <b>I Foundations and Classical Machine Learning</b>          | <b>1</b>  |
| <b>1 Introduction to Machine Learning</b>                    | <b>3</b>  |
| 1.1 Why This Chapter Comes First . . . . .                   | 4         |
| 1.2 What Is Machine Learning? . . . . .                      | 5         |
| 1.2.1 An operational definition . . . . .                    | 5         |
| 1.2.2 ML inverts the direction of programming . . . . .      | 6         |
| 1.2.3 When ML earns its keep — and when it does not . . .    | 7         |
| 1.2.4 The formal core: empirical risk minimization . . . . . | 7         |
| 1.2.5 The Bayes-optimal reference . . . . .                  | 10        |
| 1.2.6 Notation for the rest of the book . . . . .            | 11        |
| 1.3 The Machine-Learning Landscape . . . . .                 | 11        |
| 1.3.1 The four learning paradigms . . . . .                  | 11        |
| 1.3.2 Two orthogonal operational axes . . . . .              | 13        |
| 1.3.3 No free lunch, and what it licenses . . . . .          | 15        |
| 1.4 Why ML Systems Fail in Production . . . . .              | 16        |
| 1.4.1 The small green box . . . . .                          | 16        |
| 1.4.2 The notebook-to-production gap, localized . . . . .    | 17        |
| 1.5 The End-to-End ML Lifecycle . . . . .                    | 18        |
| 1.5.1 From CRISP-DM to a monitoring-driven loop . . . . .    | 18        |
| 1.5.2 It is a loop, not a waterfall . . . . .                | 19        |
| 1.6 Framing the Problem and Choosing the Metric . . . . .    | 20        |
| 1.6.1 Step 1 is where projects are won . . . . .             | 20        |
| 1.6.2 Regression metrics: how the business pays for errors   | 21        |
| 1.6.3 Classification under imbalance: why accuracy lies . .  | 21        |
| 1.7 A First Look at the Data . . . . .                       | 24        |
| 1.7.1 The raw material . . . . .                             | 24        |
| 1.7.2 A feature dictionary, declared up front . . . . .      | 24        |
| 1.7.3 What to interrogate, and why . . . . .                 | 25        |
| 1.8 Data, Splits, and Leakage . . . . .                      | 27        |

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| 1.8.1    | Splits: the discipline that protects your estimate of $R(f)$           | 27        |
| 1.8.2    | Data leakage: the silent killer of fintech models . . . .              | 28        |
| 1.9      | Baselines, Evaluation, and Generalization . . . . .                    | 30        |
| 1.9.1    | Why generalization is hard: the bias–variance decomposition . . . . .  | 30        |
| 1.9.2    | Always ship a baseline first . . . . .                                 | 31        |
| 1.9.3    | Cross-validation: cheap insurance on the estimate . .                  | 31        |
| 1.10     | Presenting and Deciding . . . . .                                      | 33        |
| 1.10.1   | The decision memo . . . . .                                            | 33        |
| 1.10.2   | Champion, challenger, and shadow mode . . . . .                        | 33        |
| 1.11     | Fairness, Explainability, and Regulation . . . . .                     | 34        |
| 1.11.1   | Making “fair” precise enough to test . . . . .                         | 34        |
| 1.11.2   | Explainability as a deployment requirement . . . . .                   | 35        |
| 1.12     | From Model to System: MLOps and the Agentic Outlook . .                | 35        |
| 1.12.1   | The MLOps spine of this course . . . . .                               | 36        |
| 1.12.2   | Where LLMs and agents fit — and where they do not                      | 37        |
| 1.13     | Chapter Summary . . . . .                                              | 38        |
|          | Exercises . . . . .                                                    | 41        |
|          | Key Terms . . . . .                                                    | 42        |
|          | Further Reading . . . . .                                              | 42        |
| <b>2</b> | <b>Data Preparation and Feature Engineering</b>                        | <b>45</b> |
| 2.1      | Introduction: From Raw Data to Model-Ready Data . . . . .              | 46        |
| 2.1.1    | The Six Dimensions of Data Quality . . . . .                           | 47        |
| 2.1.2    | The Unit of Prediction . . . . .                                       | 48        |
| 2.2      | Missing Data . . . . .                                                 | 48        |
| 2.2.1    | Rubin’s Taxonomy of Missingness . . . . .                              | 49        |
| 2.2.2    | Diagnosis Before Treatment . . . . .                                   | 50        |
| 2.2.3    | Imputation I: Simple Statistics with Indicators . . . .                | 50        |
| 2.2.4    | Imputation II: Model-Based Methods . . . . .                           | 51        |
| 2.2.5    | A Decision Framework . . . . .                                         | 52        |
| 2.3      | Outliers and Duplicates . . . . .                                      | 53        |
| 2.3.1    | Detecting Outliers . . . . .                                           | 53        |
| 2.3.2    | Treating Outliers, and When Not To . . . . .                           | 54        |
| 2.3.3    | Duplicates and Entity Resolution . . . . .                             | 54        |
| 2.4      | Scaling and Transformations . . . . .                                  | 55        |
| 2.4.1    | Why Scale, and With What . . . . .                                     | 55        |
| 2.4.2    | Taming Skew: Log and Power Transforms . . . . .                        | 56        |
| 2.5      | Encoding Categorical Variables . . . . .                               | 57        |
| 2.5.1    | Low Cardinality: One-Hot and Ordinal . . . . .                         | 57        |
| 2.5.2    | High Cardinality I: Frequency Encoding and the Hashing Trick . . . . . | 58        |
| 2.5.3    | High Cardinality II: Target Encoding . . . . .                         | 58        |
| 2.6      | Feature Engineering . . . . .                                          | 59        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 2.6.1    | Engineered versus Learned Features . . . . .                        | 59        |
| 2.6.2    | Domain Features for Mobile-Money Credit Scoring . . . . .           | 60        |
| 2.6.3    | Datetime Features and Cyclical Encoding . . . . .                   | 61        |
| 2.6.4    | Interactions, Binning, and Crosses . . . . .                        | 61        |
| 2.6.5    | What Makes a Feature Good? . . . . .                                | 62        |
| 2.7      | Data Leakage . . . . .                                              | 63        |
| 2.7.1    | Definition and Why It Is Dangerous . . . . .                        | 63        |
| 2.7.2    | Six Common Causes . . . . .                                         | 64        |
| 2.7.3    | Temporal and Group-Aware Splits . . . . .                           | 64        |
| 2.7.4    | A Detection Protocol . . . . .                                      | 65        |
| 2.7.5    | The Structural Antidote: Pipelines . . . . .                        | 66        |
| 2.8      | From Pipelines to Feature Platforms . . . . .                       | 67        |
|          | Assignment 2 (graded, 20 marks) . . . . .                           | 68        |
|          | Exercises . . . . .                                                 | 69        |
|          | Key Terms . . . . .                                                 | 70        |
|          | Further Reading . . . . .                                           | 70        |
| <b>3</b> | <b>Classical Machine Learning and Rigorous Evaluation</b> . . . . . | <b>71</b> |
| 3.1      | Introduction . . . . .                                              | 72        |
| 3.2      | The Classical Toolbox . . . . .                                     | 73        |
| 3.2.1    | Why classical models still run production tabular ML . . . . .      | 73        |
| 3.2.2    | Logistic regression . . . . .                                       | 73        |
| 3.2.3    | Softmax regression for multiclass problems . . . . .                | 75        |
| 3.2.4    | Maximum-margin classification: support vector machines . . . . .    | 75        |
| 3.2.5    | Neighbours and Bayes: two more baselines . . . . .                  | 76        |
| 3.2.6    | Choosing among the classical learners . . . . .                     | 77        |
| 3.3      | Beyond Accuracy: Metrics that Match the Problem . . . . .           | 78        |
| 3.3.1    | The accuracy trap . . . . .                                         | 78        |
| 3.3.2    | The confusion matrix . . . . .                                      | 78        |
| 3.3.3    | Precision, recall, and the $F_\beta$ family . . . . .               | 79        |
| 3.3.4    | From metrics to money: cost-sensitive evaluation . . . . .          | 79        |
| 3.4      | Ranking Quality: ROC and Precision–Recall Analysis . . . . .        | 80        |
| 3.4.1    | Scores, not labels: the threshold as a free parameter . . . . .     | 80        |
| 3.4.2    | The ROC curve and AUC . . . . .                                     | 81        |
| 3.4.3    | Precision–recall curves: the imbalance-honest view . . . . .        | 82        |
| 3.5      | Calibration: Do the Probabilities Mean Anything? . . . . .          | 82        |
| 3.6      | Cross-Validation and Honest Uncertainty . . . . .                   | 84        |
| 3.6.1    | Why one split is not enough . . . . .                               | 84        |
| 3.6.2    | Cross-validating the pipeline . . . . .                             | 85        |
| 3.6.3    | Nested cross-validation for model selection . . . . .               | 86        |
| 3.6.4    | Comparing models honestly . . . . .                                 | 86        |
| 3.7      | Learning from Imbalanced Data . . . . .                             | 87        |
| 3.7.1    | Three families of remedies . . . . .                                | 87        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 3.7.2    | SMOTE: synthesising minority examples . . . . .                         | 88        |
| 3.7.3    | The cardinal sin: resampling before splitting . . . . .                 | 89        |
| 3.8      | Case Study: Evaluation Design for the Thin-File Credit Scorer . . . . . | 90        |
| 3.9      | Chapter Summary . . . . .                                               | 91        |
| <b>4</b> | <b>Decision Trees, Ensembles and Hyperparameter Tuning</b> . . . . .    | <b>95</b> |
| 4.1      | Introduction . . . . .                                                  | 96        |
| 4.2      | Decision Trees: The Building Block . . . . .                            | 98        |
| 4.2.1    | A tree is a sequence of questions . . . . .                             | 98        |
| 4.2.2    | The same object, drawn as a partition . . . . .                         | 98        |
| 4.2.3    | How a tree is grown . . . . .                                           | 99        |
| 4.2.4    | Controlling complexity: stopping and pruning . . . . .                  | 100       |
| 4.2.5    | The flaw that motivates the rest of the chapter . . . . .               | 101       |
| 4.3      | Why Ensembles Work . . . . .                                            | 102       |
| 4.3.1    | The wisdom of independent crowds . . . . .                              | 102       |
| 4.3.2    | The variance of an average . . . . .                                    | 103       |
| 4.4      | Voting Classifiers . . . . .                                            | 104       |
| 4.5      | Bagging and Random Forests . . . . .                                    | 106       |
| 4.5.1    | Bootstrap aggregating . . . . .                                         | 106       |
| 4.5.2    | Out-of-bag evaluation . . . . .                                         | 106       |
| 4.5.3    | Random Forests: decorrelating the trees . . . . .                       | 107       |
| 4.6      | Boosting . . . . .                                                      | 109       |
| 4.6.1    | The idea, before the algebra . . . . .                                  | 109       |
| 4.6.2    | A worked trace . . . . .                                                | 110       |
| 4.6.3    | From residuals to gradients . . . . .                                   | 111       |
| 4.6.4    | AdaBoost, the historical special case . . . . .                         | 112       |
| 4.6.5    | Shrinkage and early stopping . . . . .                                  | 113       |
| 4.7      | Regularised Gradient Boosting: XGBoost, LightGBM, CatBoost . . . . .    | 113       |
| 4.7.1    | Regularisation inside the objective . . . . .                           | 114       |
| 4.7.2    | The library landscape . . . . .                                         | 115       |
| 4.7.3    | Monotonic constraints and regulated deployment . . . . .                | 115       |
| 4.8      | Stacked Generalisation . . . . .                                        | 117       |
| 4.9      | Hyperparameter Optimisation . . . . .                                   | 118       |
| 4.9.1    | What is a hyperparameter, and why is it different? . . . . .            | 118       |
| 4.9.2    | A ladder of strategies . . . . .                                        | 119       |
| 4.9.3    | Grid search versus random search . . . . .                              | 119       |
| 4.9.4    | Learning from previous trials: Bayesian search and TPE . . . . .        | 121       |
| 4.9.5    | Spending less per trial: successive halving and Hyperband . . . . .     | 122       |
| 4.9.6    | Designing the search space . . . . .                                    | 123       |
| 4.9.7    | Honest evaluation of a tuned model . . . . .                            | 123       |
| 4.10     | Case Study: Round Three of the Credit Scorer . . . . .                  | 125       |
|          | Exercises . . . . .                                                     | 127       |
|          | Key Terms . . . . .                                                     | 128       |

|                                                                   |                |
|-------------------------------------------------------------------|----------------|
| Further Reading . . . . .                                         | 129            |
| <b>5 Unsupervised Learning and Dimensionality Reduction</b>       | <b>131</b>     |
| 5.1 Introduction: Learning Without Labels . . . . .               | 132            |
| 5.2 The Unsupervised Landscape . . . . .                          | 134            |
| 5.3 Clustering with K-Means . . . . .                             | 136            |
| 5.3.1 The objective . . . . .                                     | 136            |
| 5.3.2 Choosing $k$ . . . . .                                      | 137            |
| 5.3.3 Implementation . . . . .                                    | 139            |
| 5.4 Density-Based Clustering: DBSCAN . . . . .                    | 140            |
| 5.5 Gaussian Mixtures and EM . . . . .                            | 142            |
| 5.5.1 The model . . . . .                                         | 142            |
| 5.5.2 Fitting by Expectation–Maximisation . . . . .               | 143            |
| 5.5.3 Selecting $k$ by information criteria . . . . .             | 145            |
| 5.5.4 Mixtures as anomaly detectors . . . . .                     | 146            |
| 5.6 The Curse of Dimensionality . . . . .                         | 147            |
| 5.7 Principal Component Analysis . . . . .                        | 148            |
| 5.7.1 Variance maximisation . . . . .                             | 148            |
| 5.7.2 Reconstruction-error minimisation . . . . .                 | 149            |
| 5.7.3 Computation and the choice of $d$ . . . . .                 | 149            |
| 5.7.4 Variants you will actually need . . . . .                   | 151            |
| 5.8 Case Study: The Segmentation Pipeline in Production . . . . . | 152            |
| 5.8.1 Profiling: turning clusters into personas . . . . .         | 152            |
| 5.8.2 Stability: a segment that moves is not a segment . . . . .  | 153            |
| Exercises . . . . .                                               | 156            |
| Key Terms . . . . .                                               | 157            |
| Further Reading . . . . .                                         | 158            |
| <br><b>II Deep Learning and Generative AI</b>                     | <br><b>159</b> |
| <b>6 Deep Learning Fundamentals</b>                               | <b>165</b>     |
| 6.1 Introduction . . . . .                                        | 165            |
| 6.2 Core Theory . . . . .                                         | 165            |
| 6.3 Implementation . . . . .                                      | 165            |
| <b>7 Computer Vision with Convolutional Networks</b>              | <b>167</b>     |
| 7.1 Introduction . . . . .                                        | 167            |
| 7.2 Core Theory . . . . .                                         | 167            |
| 7.3 Implementation . . . . .                                      | 167            |
| <b>8 Sequence Models and Time Series</b>                          | <b>169</b>     |
| 8.1 Introduction . . . . .                                        | 169            |
| 8.2 Core Theory . . . . .                                         | 169            |

|            |                                                     |            |
|------------|-----------------------------------------------------|------------|
| 8.3        | Implementation . . . . .                            | 169        |
| <b>9</b>   | <b>Transformers, Large Language Models and RAG</b>  | <b>171</b> |
| 9.1        | Introduction . . . . .                              | 171        |
| 9.2        | Core Theory . . . . .                               | 171        |
| 9.3        | Implementation . . . . .                            | 171        |
| <b>III</b> | <b>MLOps and Production Systems</b>                 | <b>173</b> |
| <b>10</b>  | <b>MLOps: Packaging, Serving, Tracking, CI/CD</b>   | <b>175</b> |
| 10.1       | Introduction . . . . .                              | 175        |
| 10.2       | Core Theory . . . . .                               | 175        |
| 10.3       | Implementation . . . . .                            | 175        |
| <b>11</b>  | <b>Model Monitoring, Drift and ML System Design</b> | <b>177</b> |
| 11.1       | Introduction . . . . .                              | 177        |
| 11.2       | Core Theory . . . . .                               | 177        |
| 11.3       | Implementation . . . . .                            | 177        |
|            | <b>Appendices</b>                                   | <b>181</b> |
| <b>A</b>   | <b>Capstone Project</b>                             | <b>181</b> |
| <b>B</b>   | <b>Solutions to Exercises</b>                       | <b>183</b> |
|            | Chapter 1 . . . . .                                 | 183        |
|            | Chapter 2 . . . . .                                 | 190        |
|            | Chapter 3 . . . . .                                 | 195        |

# Preface

---

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

## How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in [Appendix A](#).

| Chapter | Topic                                    | Course week |
|---------|------------------------------------------|-------------|
| 1       | End-to-End Machine Learning              | 1           |
| 2       | Data Preparation & Feature Engineering   | 2           |
| 3       | Classical ML & Rigorous Evaluation       | 3           |
| 4       | Ensemble Methods & Hyperparameter Tuning | 4           |
| 5       | Unsupervised Learning & Dim. Reduction   | 5           |
| 6       | Deep Learning Fundamentals               | 6           |
| 7       | Computer Vision with CNNs                | 7           |
| 8       | Sequence Models & Time Series            | 9           |
| 9       | Transformers, LLMs & RAG                 | 10          |
| 10      | MLOps: Serving, Tracking & CI/CD         | 11–12       |
| 11      | Monitoring, Drift & System Design        | 13          |
| A       | Capstone Project                         | 14–15       |

## Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

*Javier Serrano & Emmanuel Lee Bundi*  
*Nairobi, 2026*

## **Part I**

# **Foundations and Classical Machine Learning**





## CHAPTER 5

# Unsupervised Learning and Dimensionality Reduction

---

Course week 5

### Learning Objectives

By the end of this chapter, and of the Week 5 session it accompanies, you should be able to:

- state the unsupervised learning problem formally, and classify a business question as clustering, density estimation, dimensionality reduction or anomaly detection;
- derive Lloyd’s algorithm from the  $K$ -means objective, prove that it converges, and select  $k$  using inertia, the silhouette coefficient and information criteria rather than intuition;
- define density-reachability and apply DBSCAN to data whose clusters are non-convex, unequally sized, or contaminated with noise;
- fit a Gaussian mixture model by Expectation–Maximisation, prove that the algorithm never decreases the observed-data log-likelihood, and use the fitted density both for soft segmentation and for anomaly scoring;
- explain the curse of dimensionality quantitatively, and derive principal component analysis both as variance maximisation and as reconstruction-error minimisation, showing the two are the same problem;
- build, profile, audit and ship an unsupervised segmentation and anomaly-detection pipeline for thin-file mobile-money borrowers under Kenyan data-protection and digital-credit rules.

## 5.1 Introduction: Learning Without Labels

Every technique in [Chapters 3 and 4](#) shared a common premise: that for each training instance  $x_i$  we possessed a target  $y_i$  telling us the right answer. That premise is a luxury. Labels are produced by people, or by the slow arrival of real-world outcomes, and both are expensive. A digital lender learns whether a loan defaulted only after the loan has run its course; a fraud team learns that a transaction was fraudulent only after a customer complains. Meanwhile the transaction log grows by millions of rows a day, entirely unlabelled.

*Unsupervised learning* is the family of methods that operate on  $\mathcal{D} = \{x_1, \dots, x_m\}$  with no  $y$  at all. Instead of learning a mapping  $f : \mathcal{X} \rightarrow \mathcal{Y}$ , we learn something about the structure of the data distribution  $p(x)$  itself: where its mass concentrates (clustering, density estimation), what its in-

intrinsic dimension is (dimensionality reduction), and which observations sit implausibly far from that mass (anomaly detection).

Two motivations run through this chapter, and it is worth separating them because they lead to different design decisions.

1. **Discovery.** We want to *understand* a population we do not yet have outcomes for. A lender entering a new market segment needs to know what kinds of customer exist before it can price for them. Here the output is read by humans, so interpretability and stability dominate.
2. **Preprocessing.** We want to *compress or clean* a feature matrix before a supervised model consumes it. Here the output is read by another algorithm, so the criterion is downstream performance, and the whole thing must live inside a leakage-safe pipeline of the kind we built in [Chapter 2](#).

Confusing the two is the most common source of trouble in applied unsupervised work. A *t*-SNE map is superb for discovery and useless for preprocessing; a randomised PCA with 95% retained variance is superb for preprocessing and nearly meaningless to a credit committee. We will be explicit throughout about which job each method is for.

**Case Study: Segmenting Thin-File Borrowers (Round Four)**

Our Nairobi digital lender now has a tuned, monotone, cost-aware gradient-boosted scorecard from [Chapter 4](#), trained on roughly 180,000 historical loans with a 4.1% default rate and the asymmetric economics of KES 10,000 per missed default against KES 800 per wrongly rejected borrower. Product now asks two questions the scorecard cannot answer.

First: the wallet base is 3.4 million users, but only a small fraction have ever borrowed. What kinds of customer are in the other 95%, and which of them should be offered a first loan? There are no labels for these users at all, only 90 days of mobile-money behaviour: airtime purchases, P2P sends and receives, paybill and till payments, agent cash-in and cash-out, balance trajectories.

Second: the fraud team is drowning. SIM-swap takeovers and agent collusion produce transaction patterns that look wrong to an experienced analyst but that no supervised model has been trained to catch, because confirmed-fraud labels arrive months late and number in the hundreds, not the thousands.

Both are unsupervised problems. The first is segmentation: partition wallets into behaviourally coherent groups that the business can name, price and monitor. The second is anomaly detection: score each wallet-week by how implausible it is under a model of normal behaviour, and route the least plausible to a finite review queue. This chapter builds both, from a feature matrix  $X \in \mathbb{R}^{m \times n}$  with  $n \approx 200$  correlated behavioural aggregates and  $m \approx 150,000$  sampled wallets.

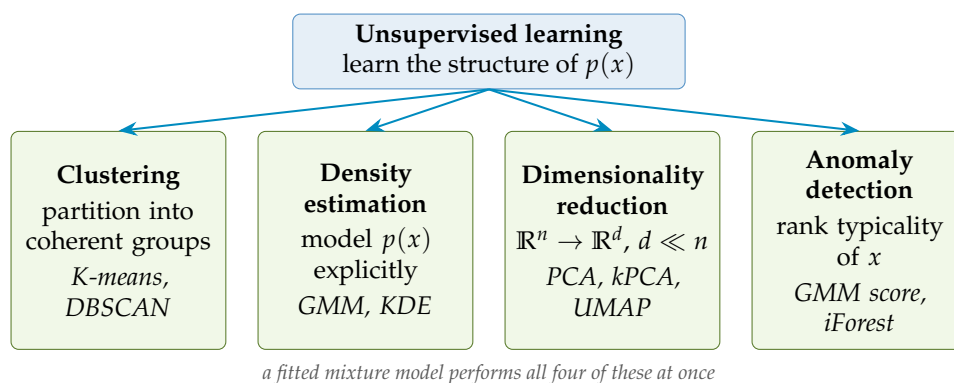
**Roadmap.** [Section 5.2](#) sets up the problem formally and maps the four task families onto one another. [Sections 5.3](#) to [5.5](#) develop the three clustering workhorses (centroid-based, density-based and probabilistic), each with its selection criterion for the number of clusters. [Section 5.6](#) explains why high dimension is hostile, motivating [Section 5.7](#) on principal component analysis and ?? on nonlinear embeddings for visualisation. [Section 5.8](#) assembles the production pipeline and its governance obligations. The chapter closes with Lab 5, Assignment 5 and exercises.

## 5.2 The Unsupervised Landscape

**Definition 5.1** (Unsupervised learning problem). Given an i.i.d. sample  $\mathcal{D} = \{x_1, \dots, x_m\}$ ,  $x_i \in \mathcal{X} \subseteq \mathbb{R}^n$ , drawn from an unknown distribution  $p$ , an unsupervised learning algorithm returns a structural summary of  $p$ : a partition  $C : \mathcal{X} \rightarrow \{1, \dots, k\}$  (clustering), a density estimate  $\hat{p}$  (density estimation), an encoder–decoder pair  $(g, h)$  with  $g : \mathbb{R}^n \rightarrow \mathbb{R}^d$ ,  $d < n$  (di-

mensionality reduction), or a score  $s : \mathcal{X} \rightarrow \mathbb{R}$  ranking typicality (anomaly detection).

The absence of  $y$  has one consequence that shapes everything else: there is no held-out loss to minimise, and therefore *no universally correct answer*. Two analysts can partition the same wallets into four segments and into nine segments, and neither is wrong in the way a miscalibrated classifier is wrong. Validation must come from three other places: internal indices that measure geometric coherence (inertia, silhouette), likelihood-based criteria that penalise complexity (AIC, BIC), and, decisive in practice, *downstream utility*, whether that is a supervised model's lift or a product manager's ability to act on a segment.



**Figure 5.1:** The four unsupervised task families. The boundaries are conventional rather than fundamental: a Gaussian mixture is at once a clustering, a density model, a soft feature transform and an anomaly scorer.

Figure 5.1 shows the four families, and it is worth stressing that they overlap heavily. A fitted mixture model gives a partition (take the arg max responsibility), a density (evaluate  $p(x)$ ), a  $k$ -dimensional soft encoding (the responsibility vector), and an anomaly score (the log density). This is not a coincidence: all four questions are questions about where probability mass lives.

#### Unsupervised outputs are inputs

An unsupervised result is almost never the deliverable. Segments become pricing tiers, credit limits and marketing lists; anomaly scores become a review queue with a fixed daily capacity; principal components become features for a supervised model. Design backwards from that consumer. If a segment cannot be named and acted upon, it does not matter that its silhouette score is high.

## 5.3 Clustering with K-Means

### 5.3.1 The objective

**Definition 5.2** (*K-means objective*). Given  $\mathcal{D} = \{x_1, \dots, x_m\} \subset \mathbb{R}^n$  and an integer  $k$ , the *K-means problem* is to find centroids  $M = \{\mu_1, \dots, \mu_k\} \subset \mathbb{R}^n$  and an assignment  $c : \{1, \dots, m\} \rightarrow \{1, \dots, k\}$  minimising the *inertia*, or within-cluster sum of squares,

$$J(M, c) = \sum_{i=1}^m \|x_i - \mu_{c(i)}\|_2^2 = \sum_{j=1}^k \sum_{i: c(i)=j} \|x_i - \mu_j\|_2^2. \quad (5.1)$$

Minimising Eq. (5.1) jointly over  $M$  and  $c$  is NP-hard for  $k \geq 2$  in general dimension, so we settle for a local optimum found by coordinate descent. Fixing  $M$ , the optimal  $c$  assigns each point to its nearest centroid; fixing  $c$ , the optimal  $\mu_j$  is the mean of its cluster, since  $\sum_{i \in C_j} \|x_i - \mu\|^2$  is minimised at  $\mu = |C_j|^{-1} \sum_{i \in C_j} x_i$ . Alternating the two is *Lloyd's algorithm* (Lloyd, 1982).

**Proposition 5.1** (Convergence of Lloyd's algorithm). *Lloyd's algorithm never increases  $J$ , and terminates after finitely many iterations at a partition that is locally optimal in the sense that no single reassignment or centroid update reduces  $J$ .*

*Proof.* Write  $J_t$  for the objective after iteration  $t$ . The assignment step replaces  $c$  by  $c'(i) = \arg \min_j \|x_i - \mu_j\|^2$  with  $M$  held fixed; since it minimises each summand of Eq. (5.1) independently,  $J(M, c') \leq J(M, c)$ . The update step replaces each  $\mu_j$  by the mean of  $C_j$  with  $c'$  held fixed. For any fixed set  $S$ ,  $\phi(\mu) = \sum_{i \in S} \|x_i - \mu\|^2$  is a strictly convex quadratic with  $\nabla \phi(\mu) = -2 \sum_{i \in S} (x_i - \mu)$ , which vanishes uniquely at  $\mu = |S|^{-1} \sum_{i \in S} x_i$ ; hence  $J(M', c') \leq J(M, c')$ . Composing,  $J_{t+1} \leq J_t$ .

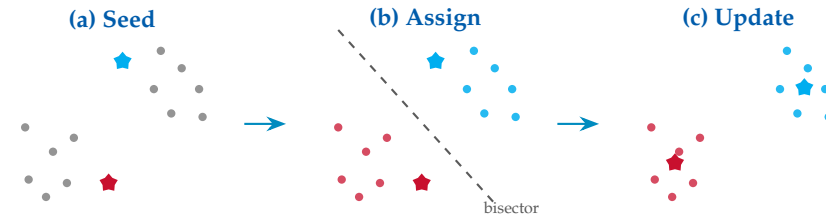
For termination, note that the algorithm's state is fully determined by the assignment  $c$ , of which there are at most  $k^m$ , a finite number. Because each iteration weakly decreases  $J$ , and because the centroid step *strictly* decreases  $J$  whenever the assignment changes and no cluster is empty, no assignment can recur without the algorithm having already stabilised. The algorithm therefore reaches a state in which  $c$  is unchanged by an assignment step, at which point both steps are at a fixed point.  $\square$

Proposition 5.1 guarantees convergence but says nothing about *quality*: the local optimum reached depends entirely on initialisation, and bad initialisations produce badly wrong answers. The standard remedy is *k-means++ seeding* (Arthur and Vassilvitskii, 2007), which chooses the first centroid uniformly at random and each subsequent centroid with probability proportional to its squared distance from the nearest already-chosen centroid.

**Proposition 5.2** (*k*-means++ approximation guarantee). Write  $J^*$  for the optimal inertia in Eq. (5.1), and  $J_{++}$  for the inertia obtained by running the seeding step of *k*-means++ alone, before any Lloyd iteration. Then

$$\mathbb{E}[J_{++}] \leq 8(\ln k + 2)J^*.$$

The proof is in Arthur and Vassilvitskii (2007) and is beyond our scope; what matters is the shape of the guarantee. Seeding alone (with no refinement) is within a logarithmic factor of optimal *in expectation*, which is why `init="k-means++"` is the scikit-learn default and why you should never override it. Combined with `n_init` restarts, keeping the lowest inertia, this makes catastrophic initialisation rare.



**Figure 5.2:** One iteration of Lloyd’s algorithm. Because assignment uses squared Euclidean distance, the boundary between any two centroids is the perpendicular bisector of the segment joining them; the induced partition is a Voronoi tessellation and every cell is convex.

The geometry in Fig. 5.2 carries the key limitation. Because the decision rule is  $\arg \min_j \|x - \mu_j\|^2$ , the boundary between clusters  $j$  and  $l$  is  $\{x : \|x - \mu_j\| = \|x - \mu_l\|\}$ , a hyperplane. *K*-means can only produce convex, roughly isotropic cells. Elongated, nested or crescent-shaped structure will be carved across, not discovered.

#### Clustering unscaled features

*K*-means minimises a Euclidean norm, so a feature measured in KES with a standard deviation of 12,000 contributes roughly  $10^6$  times more to Eq. (5.1) than a count feature with a standard deviation of 12. Clustering unscaled mobile-money data returns a partition on transaction value and nothing else, and it will look perfectly convincing. Always place a `StandardScaler` (or a rank/quantile transform for heavy-tailed monetary variables) before the clusterer, inside the same Pipeline, so the scaler is fitted on training data only.

### 5.3.2 Choosing $k$

Inertia cannot select  $k$ : Eq. (5.1) is non-increasing in  $k$  and reaches zero at  $k = m$ . Three tools are used in practice, in increasing order of rigour.

**The elbow heuristic.** Plot  $J(k)$  and look for the point after which additional clusters buy little variance reduction. It is fast and coarse; use it to bracket a candidate range, not to decide.

**The silhouette coefficient.** A per-instance measure of whether a point sits comfortably in its assigned cluster (Rousseeuw, 1987).

**Definition 5.3** (Silhouette coefficient). For instance  $i$  in cluster  $C_{c(i)}$  with  $|C_{c(i)}| > 1$ , let

$$a(i) = \frac{1}{|C_{c(i)}| - 1} \sum_{\substack{l \in C_{c(i)} \\ l \neq i}} d(x_i, x_l), \quad b(i) = \min_{j \neq c(i)} \frac{1}{|C_j|} \sum_{l \in C_j} d(x_i, x_l), \quad (5.2)$$

the mean distance to its own cluster and to the nearest *other* cluster. The silhouette coefficient is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1], \quad (5.3)$$

with  $s(i) = 0$  by convention when  $|C_{c(i)}| = 1$ . The *silhouette score* of a clustering is  $\bar{s} = m^{-1} \sum_i s(i)$ .

Values near +1 indicate a point deep inside a well-separated cluster; near 0, a point on a boundary; negative, a point that is on average closer to a different cluster than to its own, i.e. probably misassigned. Choosing  $k$  near the maximum of  $\bar{s}$  is standard, but the *silhouette diagram* (per-cluster sorted  $s(i)$  profiles) is more informative, because it exposes the case where one huge coherent cluster inflates  $\bar{s}$  while three small clusters are worthless. Computing  $\bar{s}$  exactly costs  $\mathcal{O}(m^2)$  distance evaluations, so subsample for large  $m$ .

**Example 5.1** (Selecting  $k$  for the wallet base). On a 20,000-wallet subsample of the case-study data, standardised and reduced to 32 principal components, a sweep over  $k$  gives the values in Table 5.1 (illustrative figures, to be regenerated from your own lab run).

The elbow sits somewhere between  $k = 4$  and  $k = 5$ ; the silhouette and BIC both point at 5. Note that all silhouette values are modest (around 0.3), which is entirely normal for real behavioural data. Textbook illustrations using well-separated synthetic blobs routinely exceed 0.7; a practitioner who demands that from mobile-money aggregates will conclude, wrongly, that there is no structure at all.

**Business admissibility.** The final filter is not statistical. Five segments can be priced, staffed, monitored and explained to a regulator; forty cannot.

**Table 5.1:** Cluster-count diagnostics on the wallet subsample. Inertia falls monotonically; the silhouette peaks at  $k = 5$ ; BIC for a full-covariance Gaussian mixture agrees. *Illustrative values.*

| $k$ | Inertia $J(k)$ ( $\times 10^5$ ) | Silhouette $\bar{s}$ | GMM BIC ( $\times 10^6$ ) |
|-----|----------------------------------|----------------------|---------------------------|
| 2   | 5.81                             | 0.244                | 3.41                      |
| 3   | 4.62                             | 0.271                | 3.28                      |
| 4   | 3.95                             | 0.302                | 3.19                      |
| 5   | 3.58                             | <b>0.318</b>         | <b>3.15</b>               |
| 6   | 3.37                             | 0.309                | 3.16                      |
| 8   | 3.09                             | 0.281                | 3.22                      |
| 12  | 2.78                             | 0.243                | 3.34                      |

If the silhouette prefers  $k = 17$  and the product team can act on five, cluster at  $k = 17$  for diagnostic purposes and then merge, but ship five.

#### A defensible procedure for choosing $k$

1. Scale (and usually reduce) features first, inside a pipeline fitted on training data only.
2. Sweep  $k$  over a plausible range; record inertia, silhouette, and (if you are fitting a mixture) BIC.
3. Use the elbow to bracket, the silhouette or BIC to choose, and a silhouette diagram to check that no single cluster is carrying the score.
4. Check *stability*: re-fit on bootstrap resamples or on an adjacent time window and measure agreement with the adjusted Rand index. An unstable partition is not a finding.
5. Apply the business veto, and document why the shipped  $k$  differs from the statistical optimum if it does.

### 5.3.3 Implementation

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

def build_segmenter(k, seed=42):
    """Scale -> PCA(95% variance) -> K-means, as one fitted artefact."""
    return Pipeline([
```

```

        ("scale", StandardScaler()),
        ("pca", PCA(n_components=0.95, random_state=seed)),
        ("km", KMeans(n_clusters=k, init="k-means++",
                      n_init=10, random_state=seed)),
    ])

rows = []
for k in range(2, 13):
    seg = build_segmenter(k).fit(X_train)
    labels = seg.named_steps["km"].labels_
    Z = seg[:-1].transform(X_train)  # PCA coordinates
    rows.append((k,
                 seg.named_steps["km"].inertia_,
                 silhouette_score(Z, labels, sample_size=20_000,
                                 random_state=0)))

for k, inertia, sil in rows:
    print(f"k={k:2d}  inertia={inertia:12.1f}  silhouette={sil:.3f}")

```

**Listing 5.1:** Leakage-safe segmentation pipeline with a cluster-count sweep. The scaler and reducer are fitted on training wallets only; the same fitted object later serves new wallets.

Two details in Listing 5.1 matter more than they look. First, the silhouette is computed in the *PCA space the clusterer actually used*, not in raw feature space: scoring a partition in a different metric than the one that produced it is meaningless. Second, `n_init=10` performs ten independent *k*-means++ restarts and keeps the lowest-inertia result; for *m* in the millions, swap `KMeans` for `MiniBatchKMeans`, which trades a small increase in inertia for an order-of-magnitude speed-up.

## 5.4 Density-Based Clustering: DBSCAN

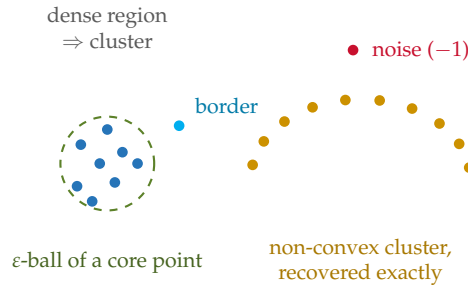
*K*-means asks “which centroid is nearest?”. DBSCAN (Ester et al., 1996) asks a different question: “is this region crowded?”. Clusters become maximal connected regions of high density, which frees them from any shape assumption and (crucially for fraud work) lets the algorithm refuse to assign a point at all.

**Definition 5.4** (Core, border and noise points). Fix a radius  $\varepsilon > 0$  and an integer  $\text{minPts} \geq 1$ . Let  $N_\varepsilon(x) = \{x' \in \mathcal{D} : d(x, x') \leq \varepsilon\}$  be the  $\varepsilon$ -neighbourhood of  $x$ . Then  $x$  is

- a *core point* if  $|N_\varepsilon(x)| \geq \text{minPts}$ ;
- a *border point* if it is not core but lies in  $N_\varepsilon(x')$  for some core point  $x'$ ;
- *noise* otherwise.

A point  $q$  is *directly density-reachable* from a core point  $p$  if  $q \in N_\varepsilon(p)$ ; *density-reachable* is the transitive closure of that relation over core points; and  $p, q$

are *density-connected* if some core point reaches both. A DBSCAN cluster is a maximal density-connected set.



**Figure 5.3:** DBSCAN vocabulary. A single radius  $\epsilon$  and a count threshold  $\text{minPts}$  determine core, border and noise points; clusters are maximal density-connected sets and may take any shape.

**Proposition 5.3** (Determinism up to border assignment). *For fixed  $\epsilon$  and  $\text{minPts}$ , the set of core points and the partition of core points into clusters produced by DBSCAN are independent of the order in which points are visited. Only the cluster label of a border point that is density-reachable from two or more clusters may depend on that order.*

*Proof.* Being core is a property of  $|N_\epsilon(x)|$  alone, hence order-independent. Restrict attention to core points and define the graph  $G$  whose vertices are core points and whose edges join pairs at distance  $\leq \epsilon$ . Density-reachability restricted to core points is exactly reachability in  $G$ , so density-connectedness is the connected-component relation on  $G$ , which is a property of  $G$  and therefore order-independent. A border point  $q$  belongs to no such component; it is assigned to whichever cluster of a core point  $p$  with  $q \in N_\epsilon(p)$  is encountered first. If exactly one component contains such a  $p$ , the assignment is forced; if two or more do, it is not.  $\square$

Proposition 5.3 is the reason DBSCAN is reproducible in practice despite having no random seed: the ambiguity is confined to points that are, by construction, on a boundary and therefore of low business consequence.

**Choosing  $\epsilon$  and  $\text{minPts}$ .** Set  $\text{minPts}$  first, from domain reasoning: it is the smallest group you are willing to call a cluster, and a common default is  $2n$  for  $n$ -dimensional data, or simply 10–25 after dimensionality reduction. Then plot the sorted distance from each point to its  $\text{minPts}$ -th nearest neighbour (the *k-distance graph*) and take  $\epsilon$  at the knee, where the curve turns sharply upward. Points beyond the knee are the ones the algorithm will call noise.

```

import numpy as np
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

min_pts = 15
nn = NearestNeighbors(n_neighbors=min_pts).fit(Z_train)  # Z = PCA space
dists, _ = nn.kneighbors(Z_train)
kdist = np.sort(dists[:, -1])  # distance to the 15th neighbour

# Knee = point of maximum curvature; a robust proxy is the largest
# second difference over the upper tail of the sorted curve.
tail = kdist[int(0.5 * len(kdist)):]
eps = tail[np.argmax(np.diff(tail, n=2)) + 1]
print(f"chosen eps = {eps:.3f}")

db = DBSCAN(eps=eps, min_samples=min_pts, n_jobs=-1).fit(Z_train)
labels = db.labels_  # -1 == noise
n_clusters = len(set(labels) - {-1})
print(f"{n_clusters} clusters, {(labels == -1).mean():.2%} noise")

```

**Listing 5.2:** DBSCAN with a data-driven  $\varepsilon$  from the  $k$ -distance knee, applied to PCA-reduced wallet features.

DBSCAN has two operational weaknesses worth stating plainly. First, a single global  $\varepsilon$  cannot serve clusters of very different densities: tune for the sparse cluster and the dense ones merge; tune for the dense ones and the sparse cluster dissolves into noise. HDBSCAN (Campello et al., 2013) addresses this by building a hierarchy over all  $\varepsilon$  and extracting the most stable clusters, and is the right default for exploratory work today. Second, scikit-learn’s DBSCAN exposes no predict method, because the algorithm is defined on a fixed dataset. In production, either train a lightweight classifier (a  $k$ -NN on the core points) on the DBSCAN labels to score new wallets, or re-fit on a schedule and version the result.

## 5.5 Gaussian Mixture Models and Expectation–Maximisation

Both algorithms so far return a hard partition and no notion of probability. For a lender that must answer “how confident are we that this wallet belongs to the salaried segment?” and “how unusual is this week’s behaviour?”, a probabilistic model is worth its extra cost.

### 5.5.1 The model

**Definition 5.5** (Gaussian mixture model). A Gaussian mixture model with  $k$  components asserts that

$$p(x \mid \theta) = \sum_{j=1}^k \pi_j \mathcal{N}(x \mid \mu_j, \Sigma_j), \quad \pi_j \geq 0, \quad \sum_{j=1}^k \pi_j = 1, \quad (5.4)$$

**Table 5.2:** Choosing a clustering algorithm by data geometry and product need.

|                      | <b>K-means</b>     | <b>DBSCAN</b>            | <b>Gaussian mixture</b>      |
|----------------------|--------------------|--------------------------|------------------------------|
| Cluster shape        | convex, isotropic  | arbitrary                | elliptical (any $\Sigma_j$ ) |
| Assignment           | hard               | hard, plus noise         | soft (responsibilities)      |
| Number of clusters   | fixed as $k$       | emergent from $\epsilon$ | fixed as $k$ , chosen by BIC |
| Outliers             | none; all assigned | explicit noise label     | low log-density              |
| Scales to $10^7$     | yes (MiniBatch)    | with spatial index       | moderately                   |
| Serves new points    | predict            | no native support        | predict_proba                |
| Typical fin-tech use | pricing segments   | fraud-ring covery        | anomaly scoring, soft tiers  |

with  $\theta = \{\pi_j, \mu_j, \Sigma_j\}_{j=1}^k$ . Equivalently, each  $x_i$  is generated by drawing a latent component  $z_i \sim \text{Categorical}(\pi)$  and then  $x_i \mid z_i = j \sim \mathcal{N}(\mu_j, \Sigma_j)$ .

Clustering is then posterior inference about  $z_i$ . By Bayes' rule the *responsibility* of component  $j$  for observation  $i$  is

$$\gamma_{ij} = p(z_i = j \mid x_i, \theta) = \frac{\pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}{\sum_{l=1}^k \pi_l \mathcal{N}(x_i \mid \mu_l, \Sigma_l)}. \quad (5.5)$$

Free covariances  $\Sigma_j$  buy exactly what K-means lacked: clusters that are elliptical, of different volumes, and differently oriented. The price is parameters. A full-covariance mixture has  $k \left( n + \frac{n(n+1)}{2} \right) + k - 1$  free parameters, which for  $n = 200$  and  $k = 5$  exceeds 100,000, another reason to reduce dimension first (Section 5.7). scikit-learn's `covariance_type` offers the ladder `full`  $\rightarrow$  `tied`  $\rightarrow$  `diag`  $\rightarrow$  `spherical`, trading flexibility for parameter count down to  $\mathcal{O}(kn)$ .

### 5.5.2 Fitting by Expectation–Maximisation

The log-likelihood  $\ell(\theta) = \sum_{i=1}^m \log \sum_{j=1}^k \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)$  has a logarithm outside a sum and admits no closed-form maximiser. The Expectation–Maximisation algorithm (Dempster et al., 1977) instead optimises a lower bound.

For any set of distributions  $q_i$  over  $\{1, \dots, k\}$ , Jensen's inequality gives

$$\begin{aligned}\ell(\theta) &= \sum_i \log \sum_j q_i(j) \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{q_i(j)} \\ &\geq \sum_i \sum_j q_i(j) \log \frac{\pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{q_i(j)} =: \mathcal{L}(q, \theta),\end{aligned}\tag{5.6}$$

the evidence lower bound (ELBO). EM alternates maximising  $\mathcal{L}$  in  $q$  (the E-step) and in  $\theta$  (the M-step):

**E-step.** With  $\theta$  fixed,  $\mathcal{L}(q, \theta) = \ell(\theta) - \sum_i \text{KL}(q_i \| p(z_i | x_i, \theta))$ , so the bound is tight and maximal at  $q_i(j) = \gamma_{ij}$  from [Eq. \(5.5\)](#).

**M-step.** With  $q$  fixed at  $\gamma$ , maximising  $\mathcal{L}$  over  $\theta$  is a weighted maximum-likelihood problem with closed-form solution. Writing  $m_j = \sum_{i=1}^m \gamma_{ij}$  for the effective count of component  $j$ ,

$$\pi_j \leftarrow \frac{m_j}{m}, \quad \mu_j \leftarrow \frac{1}{m_j} \sum_{i=1}^m \gamma_{ij} x_i, \tag{5.7}$$

$$\Sigma_j \leftarrow \frac{1}{m_j} \sum_{i=1}^m \gamma_{ij} (x_i - \mu_j)(x_i - \mu_j)^\top. \tag{5.8}$$

**Proposition 5.4** (Monotonicity of EM). *Let  $\theta^{(t)}$  be the parameters after  $t$  EM iterations. Then  $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$ , with equality only if  $\theta^{(t)}$  is a stationary point of  $\ell$ .*

*Proof.* Write  $\mathcal{L}(q, \theta)$  for the ELBO in [Eq. \(5.6\)](#). For any  $\theta$  and any  $q$ ,

$$\ell(\theta) = \mathcal{L}(q, \theta) + \sum_{i=1}^m \text{KL}(q_i \| p(z_i | x_i, \theta)),$$

which follows by expanding the KL term and using  $p(z_i | x_i, \theta) = p(x_i, z_i | \theta) / p(x_i | \theta)$ . Since KL divergence is non-negative and vanishes exactly when its arguments agree, the E-step choice  $q_i^{(t)} = p(z_i | x_i, \theta^{(t)})$  gives  $\mathcal{L}(q^{(t)}, \theta^{(t)}) = \ell(\theta^{(t)})$ . The M-step then chooses  $\theta^{(t+1)} = \arg \max_{\theta} \mathcal{L}(q^{(t)}, \theta)$ , so  $\mathcal{L}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{L}(q^{(t)}, \theta^{(t)})$ . Combining, and using non-negativity of KL once more,

$$\ell(\theta^{(t+1)}) \geq \mathcal{L}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{L}(q^{(t)}, \theta^{(t)}) = \ell(\theta^{(t)}).$$

Equality throughout forces  $\theta^{(t+1)}$  to leave the bound  $\mathcal{L}(q^{(t)}, \cdot)$  unchanged. When the bound is tight we have  $\nabla_{\theta} \mathcal{L}(q^{(t)}, \theta)|_{\theta^{(t)}} = \nabla_{\theta} \ell(\theta)|_{\theta^{(t)}}$ , so  $\theta^{(t)}$  is then a stationary point of  $\ell$ .  $\square$

Because  $\ell$  is bounded above on any region where covariances are bounded away from singularity, Proposition 5.4 implies convergence of the likelihood sequence, to a local optimum, exactly as with Lloyd's algorithm, and with the same remedy of multiple restarts (`n_init`).

**Proposition 5.5** (*K-means as a limiting case*). Fix  $\pi_j = 1/k$  and  $\Sigma_j = \sigma^2 I$  for all  $j$ . As  $\sigma^2 \rightarrow 0^+$ , the responsibilities in Eq. (5.5) converge to hard indicators  $\gamma_{ij} \rightarrow \mathbb{1}\{j = \arg \min_l \|x_i - \mu_l\|\}$  (for  $x_i$  with a unique nearest centroid), and the M-step Eq. (5.7) reduces to the K-means centroid update.

*Proof.* With equal weights and isotropic equal covariances, the Gaussian normalising constants cancel in Eq. (5.5), leaving

$$\begin{aligned} \gamma_{ij} &= \frac{\exp(-\|x_i - \mu_j\|^2 / 2\sigma^2)}{\sum_l \exp(-\|x_i - \mu_l\|^2 / 2\sigma^2)} \\ &= \frac{1}{\sum_l \exp(-(\|x_i - \mu_l\|^2 - \|x_i - \mu_j\|^2) / 2\sigma^2)}. \end{aligned}$$

Let  $j^*$  be the unique minimiser of  $\|x_i - \mu_l\|$ . For  $j = j^*$  every exponent in the denominator is  $\leq 0$  and strictly negative for  $l \neq j^*$ , so as  $\sigma^2 \rightarrow 0^+$  each such term tends to 0 and  $\gamma_{ij^*} \rightarrow 1$ . For  $j \neq j^*$  the term with  $l = j^*$  has a strictly positive exponent that diverges, so  $\gamma_{ij} \rightarrow 0$ . Substituting hard indicators into Eq. (5.7) gives  $m_j = |C_j|$  and  $\mu_j = |C_j|^{-1} \sum_{i \in C_j} x_i$ , the K-means update.  $\square$

Proposition 5.5 is more than an elegance. It tells you exactly what you assume when you reach for K-means: equal-sized, equal-variance, spherical clusters with infinitely confident assignments. When those assumptions are wrong, a mixture is not a fancier alternative: it is the correct model, and K-means is the approximation.

### 5.5.3 Selecting $k$ by information criteria

Unlike inertia, the likelihood of a mixture can be penalised for complexity and then genuinely minimised. With  $p$  free parameters and maximised likelihood  $\hat{L}$ ,

$$\text{BIC} = p \ln m - 2 \ln \hat{L}, \quad \text{AIC} = 2p - 2 \ln \hat{L}, \quad (5.9)$$

and we choose the  $(k, \text{covariance\_type})$  pair minimising the criterion (Schwarz, 1978). BIC's penalty grows with  $m$  (since  $\ln m > 2$  for  $m > 7$ ), so it prefers simpler models and is the usual choice when the segments will be read by humans; AIC targets predictive fit and tends to select more components.

**Likelihood degeneracy**

The Gaussian mixture likelihood is *unbounded*: place  $\mu_j$  exactly on a data point and shrink  $\Sigma_j \rightarrow 0$ , and  $\ell \rightarrow \infty$ . EM can and does drift into these spikes, especially with full covariances, few points per component, and high dimension. The symptom is a component with an effective count  $m_j$  of one or two and an absurd likelihood. The remedies are regularisation (scikit-learn's `reg_covar` adds a small ridge to the diagonal; do not set it to zero), constraining `covariance_type`, reducing dimension first, and restarting with `n_init`. A “best BIC” model containing a singleton component should be rejected, not shipped.

**5.5.4 Mixtures as anomaly detectors**

Because Eq. (5.4) is an actual density,  $\log p(x)$  is a principled typicality score, and thresholding it produces a review queue.

```
import numpy as np
from sklearn.mixture import GaussianMixture

best, best_bic = None, np.inf
for k in range(2, 13):
    for cov in ("full", "tied", "diag"):
        gm = GaussianMixture(n_components=k, covariance_type=cov,
                             reg_covar=1e-6, n_init=5,
                             random_state=42).fit(Z_train)
        bic = gm.bic(Z_train)
        if bic < best_bic and gm.weights_.min() * len(Z_train) > 50:
            best, best_bic = gm, bic          # reject singleton components

log_dens = best.score_samples(Z_new)        # log p(x), higher = typical

# The analyst team can review 400 cases per day: make the threshold a
# capacity decision, not an arbitrary percentile.
DAILY_CAPACITY = 400
threshold = np.partition(log_dens, DAILY_CAPACITY)[DAILY_CAPACITY]
queue = np.where(log_dens < threshold)[0]

resp = best.predict_proba(Z_new)           # soft segment membership
```

**Listing 5.3:** Fitting a mixture by BIC and converting its log-density into a capacity-constrained fraud review queue.

Note what Listing 5.3 does *not* do: it does not claim that the flagged wallets are fraudulent. It claims they are improbable under a model of normal behaviour, which is a much weaker and much more defensible statement. The threshold is set by analyst capacity, exactly as the classification threshold in Chapter 3 was set by the KES 10,000 / KES 800 cost ratio: in both cases an operational constraint, not a statistical convention, fixes the operating point. As confirmed-fraud labels accumulate from the queue, the problem graduates to supervised detection, a progression from unsupervised to supervised as labelling budget allows.

## 5.6 The Curse of Dimensionality

Every clustering method above relies on distances being informative. In high dimension they stop being so, and the failure is quantitative rather than vague.

**Proposition 5.6** (Distance concentration). *Let  $x^{(1)}, \dots, x^{(m)}, q$  be independent random vectors in  $\mathbb{R}^n$  whose coordinates are i.i.d. with finite fourth moment, and let  $D_n^{(i)} = \|x^{(i)} - q\|_2$ . Then*

$$\frac{\text{Var}[D_n^2]}{(\mathbb{E}[D_n^2])^2} = \mathcal{O}\left(\frac{1}{n}\right) \implies \frac{D_{\max,n} - D_{\min,n}}{D_{\min,n}} \xrightarrow{p} 0 \quad \text{as } n \rightarrow \infty,$$

where  $D_{\max,n}$  and  $D_{\min,n}$  are the largest and smallest of the  $m$  distances.

*Proof sketch.* Write  $D_n^2 = \sum_{t=1}^n \delta_t$  where  $\delta_t = (x_t - q_t)^2$  are i.i.d. with mean  $\alpha$  and variance  $\beta < \infty$ . Then  $\mathbb{E}[D_n^2] = n\alpha$  and  $\text{Var}[D_n^2] = n\beta$ , so the relative variance is  $\beta/(n\alpha^2) \rightarrow 0$ . Hence  $D_n^2/n \rightarrow \alpha$  in probability; by Slutsky and the continuous mapping theorem  $D_n/\sqrt{n} \rightarrow \sqrt{\alpha}$  for each of the  $m$  points simultaneously (a union bound over fixed  $m$  suffices). Therefore  $D_{\max,n}/D_{\min,n} \rightarrow 1$  in probability, which is the claim. The full argument, including the general  $L_p$  case and the conditions under which the conclusion *fails* (most importantly when the coordinates are strongly dependent), is given by [Beyer et al. \(1999\)](#).  $\square$

The practical reading of Proposition 5.6 is stark: in high enough dimension, with independent features, every point is roughly equidistant from every other. “Nearest neighbour” becomes meaningless, and with it  $k$ -NN, DBSCAN’s  $\varepsilon$ -balls, and  $K$ -means’ assignment step. A second, more familiar face of the same phenomenon is sampling density: covering  $[0, 1]^n$  at resolution 0.1 per axis requires  $10^n$  points, so the sample size needed to maintain constant local density grows exponentially in  $n$ .

The escape hatch is the final clause of the proof sketch. Real features are *not* independent. Wallet behaviour has perhaps twenty underlying drivers (income regularity, transaction velocity, merchant mix, network size) expressed through two hundred correlated measurements. The data therefore concentrates near a low-dimensional structure, and the entire business of [Section 5.7](#) and ?? is to find it.

**Two routes down**

**Projection** assumes the data lies near a flat  $d$ -dimensional *subspace* and projects onto it. It is linear, fast, approximately invertible and interpretable through its loadings: this is PCA. **Manifold learning** assumes the data lies near a *curved*  $d$ -dimensional sheet embedded in  $\mathbb{R}^n$ ; the canonical picture is a Swiss roll, which projection would squash but which can be unrolled. It is nonlinear, more expensive, and usually not invertible: this is kernel PCA, LLE, Isomap,  $t$ -SNE and UMAP.

## 5.7 Principal Component Analysis

PCA (Pearson, 1901; Hotelling, 1933) is the oldest and still the most useful dimensionality reduction method. It admits two definitions that sound different and turn out to be the same problem, and understanding that equivalence is what makes the method safe to use.

Throughout, assume the data has been centred,  $\bar{x} = m^{-1} \sum_i x_i = 0$ , so that the sample covariance is  $S = \frac{1}{m} X^\top X$  with  $X \in \mathbb{R}^{m \times n}$  the data matrix.

### 5.7.1 Variance maximisation

**Theorem 5.1** (Principal components). *Let  $S$  have eigenvalues  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$  with orthonormal eigenvectors  $v_1, \dots, v_n$ . Then for each  $d \leq n$ , the orthonormal set  $\{w_1, \dots, w_d\}$  maximising the retained variance  $\sum_{r=1}^d w_r^\top S w_r$  is  $\{v_1, \dots, v_d\}$ , and the maximum value is  $\sum_{r=1}^d \lambda_r$ .*

*Proof.* Take  $d = 1$  first. We maximise  $f(w) = w^\top S w$  subject to  $w^\top w = 1$ . The Lagrangian is  $\mathcal{L}(w, \lambda) = w^\top S w - \lambda(w^\top w - 1)$ , and  $\nabla_w \mathcal{L} = 2Sw - 2\lambda w = 0$  gives  $Sw = \lambda w$ : every stationary point is an eigenvector. At such a point  $f(w) = w^\top (\lambda w) = \lambda$ , so the maximum over the unit sphere is attained at  $w = v_1$  with value  $\lambda_1$ .

Now proceed inductively. Suppose  $w_1 = v_1, \dots, w_{r-1} = v_{r-1}$  have been fixed and we maximise  $w^\top S w$  over unit  $w$  orthogonal to all of them. Expand  $w = \sum_{t=1}^n a_t v_t$  in the eigenbasis; orthogonality forces  $a_1 = \dots = a_{r-1} = 0$ , and  $\|w\| = 1$  gives  $\sum_{t \geq r} a_t^2 = 1$ . Then

$$w^\top S w = \sum_{t \geq r} a_t^2 \lambda_t \leq \lambda_r \sum_{t \geq r} a_t^2 = \lambda_r,$$

with equality at  $w = v_r$ . Summing the  $d$  steps gives the stated maximum  $\sum_{r \leq d} \lambda_r$ . (That the greedy sequence is globally optimal over all  $d$ -dimensional subspaces follows from the Ky Fan / Courant–Fischer characterisation: for any orthonormal  $W_d$ ,  $\text{tr}(W_d^\top S W_d) \leq \sum_{r \leq d} \lambda_r$ .)  $\square$

The projection of the data onto the top  $d$  components is

$$X_{d\text{-proj}} = XW_d, \quad W_d = [v_1 \cdots v_d] \in \mathbb{R}^{n \times d}, \quad (5.10)$$

and since  $\text{tr}(S) = \sum_t \lambda_t$  is the total variance, the *explained variance ratio* of component  $r$  is  $\lambda_r / \sum_t \lambda_t$ .

### 5.7.2 Reconstruction-error minimisation

**Proposition 5.7** (Equivalence of the two formulations). *Among all rank- $d$  orthogonal projections  $P = WW^\top$  with  $W \in \mathbb{R}^{n \times d}$ ,  $W^\top W = I_d$ , the one minimising the mean squared reconstruction error*

$$E(W) = \frac{1}{m} \sum_{i=1}^m \|x_i - WW^\top x_i\|_2^2 \quad (5.11)$$

is the same  $W_d$  that maximises retained variance in Theorem 5.1.

*Proof.* Because  $P = WW^\top$  is an orthogonal projection,  $x$  decomposes orthogonally as  $x = Px + (I - P)x$ , so by Pythagoras  $\|x\|^2 = \|Px\|^2 + \|x - Px\|^2$ . Averaging over the sample,

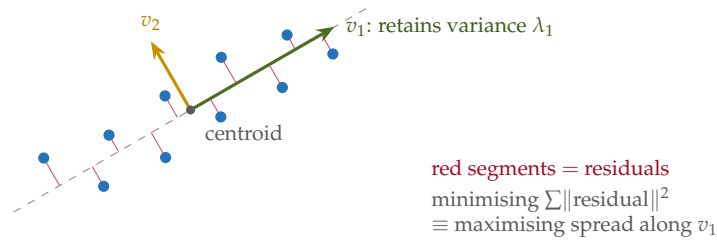
$$\frac{1}{m} \sum_i \|x_i\|^2 = \underbrace{\frac{1}{m} \sum_i \|W^\top x_i\|^2}_{\text{retained variance}} + \underbrace{E(W)}_{\text{reconstruction error}},$$

where we used  $\|Px\| = \|WW^\top x\| = \|W^\top x\|$  for orthonormal  $W$ . The left-hand side does not depend on  $W$ . Moreover  $\frac{1}{m} \sum_i \|W^\top x_i\|^2 = \text{tr}(W^\top SW)$ . Hence minimising  $E(W)$  is exactly maximising  $\text{tr}(W^\top SW)$ , whose solution is  $W_d$  by Theorem 5.1, and the minimal error is  $\sum_{t>d} \lambda_t$ .  $\square$

Proposition 5.7 has an immediate practical payoff: variance kept is error avoided, and the discarded eigenvalues  $\sum_{t>d} \lambda_t$  are exactly the information destroyed. It also hands us a free anomaly detector. If a wallet's behaviour is well described by the twenty or so latent drivers the subspace captures, its reconstruction error will be small; if not, it will not be. Per-instance reconstruction error is a legitimate typicality score, and it catches a different failure mode than the mixture density of Listing 5.3: the mixture flags points in low-density regions of the subspace, reconstruction error flags points *off* the subspace entirely.

### 5.7.3 Computation and the choice of $d$

In practice PCA is computed from the singular value decomposition  $X = U\Sigma V^\top$  rather than by forming  $S$  explicitly: the columns of  $V$  are the eigenvectors of  $X^\top X$  and the singular values satisfy  $\lambda_r = \sigma_r^2/m$ . Forming  $S$  squares the condition number and costs  $\mathcal{O}(mn^2)$ ; randomised SVD (Halko et al., 2011) obtains the top  $d$  components in roughly  $\mathcal{O}(mnd)$  and is what scikit-learn selects automatically for large problems.



**Figure 5.4:** The two faces of PCA. Maximising the variance of the projections along  $v_1$  (green) is the same optimisation as minimising the summed squared lengths of the perpendicular residuals (red), by Proposition 5.7.

**Example 5.2** (A two-feature calculation). Two standardised wallet features (P2P send count and till-payment count) have sample covariance

$$S = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}.$$

The characteristic polynomial is  $(4 - \lambda)^2 - 4 = 0$ , giving  $\lambda_1 = 6$ ,  $\lambda_2 = 2$  with eigenvectors  $v_1 = \frac{1}{\sqrt{2}}(1, 1)^\top$  and  $v_2 = \frac{1}{\sqrt{2}}(1, -1)^\top$ . Total variance is  $\text{tr}(S) = 8$ , so the first component explains  $6/8 = 75\%$  and the second 25%. The interpretation is direct:  $v_1$  is overall transaction *activity* (both counts rise together) and  $v_2$  is the *mix* between sending money and paying merchants. Projecting onto  $v_1$  alone incurs mean squared reconstruction error  $\lambda_2 = 2$ , by Proposition 5.7.

To choose  $d$  on real data, plot the cumulative explained variance  $\sum_{r \leq d} \lambda_r / \sum_t \lambda_t$  and take the smallest  $d$  reaching a threshold; 95% is the conventional default when PCA is a preprocessing step, and  $d \in \{2, 3\}$  when the purpose is plotting.

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

reducer = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=0.95, svd_solver="auto", random_state=42)),
])
Z_train = reducer.fit_transform(X_train)  # FIT on train only
Z_test = reducer.transform(X_test)        # never re-fit downstream

pca = reducer.named_steps["pca"]
print(f"{pca.n_components_} components retain "
      f"{pca.explained_variance_ratio_.sum():.1%} of variance")

# Reconstruction error: distance from the learned subspace.
Xs_test = reducer.named_steps["scale"].transform(X_test)
recon = pca.inverse_transform(Z_test)
err = np.square(Xs_test - recon).sum(axis=1)
```

```

off_manifold = err > np.quantile(err, 0.995)  # candidate anomalies

# Which raw features drive component 1? (loadings, in scaled units)
loadings = pd.Series(pca.components_[0], index=X_train.columns)
print(loadings.abs().nlargest(8))

```

**Listing 5.4:** PCA inside a leakage-safe pipeline, with reconstruction error as a second anomaly channel.

### Fitting the transformer on all the data

StandardScaler and PCA are *fitted* transformers: they estimate means, variances and eigenvectors from data. Calling `fit_transform` on the full matrix before splitting leaks test-set structure into training, and the resulting cross-validation scores are optimistic in a way that no amount of later care repairs. The rule from [Chapter 2](#) is unchanged: every fitted step lives inside the Pipeline, and the Pipeline is what gets cross-validated.

## 5.7.4 Variants you will actually need

**Table 5.3:** PCA variants and when each earns its place.

| Variant         | What it changes                                                                | Reach for it when...                                                      |
|-----------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Randomised PCA  | stochastic range-finder SVD, $\mathcal{O}(mnd)$ instead of $\mathcal{O}(mn^2)$ | $n$ is large and $d \ll n$ ; the default heuristic                        |
| Incremental PCA | mini-batch fitting via <code>partial_fit</code>                                | the matrix exceeds memory, or features stream in                          |
| Kernel PCA      | components in an implicit feature space via a kernel                           | structure is curved and $m$ is moderate ( $\mathcal{O}(m^2)$ Gram matrix) |
| Truncated SVD   | omits centring; operates on sparse matrices                                    | one-hot, TF-IDF, or any sparse design matrix                              |
| Sparse PCA      | $\ell_1$ penalty on loadings                                                   | you must explain each component with a handful of named features          |

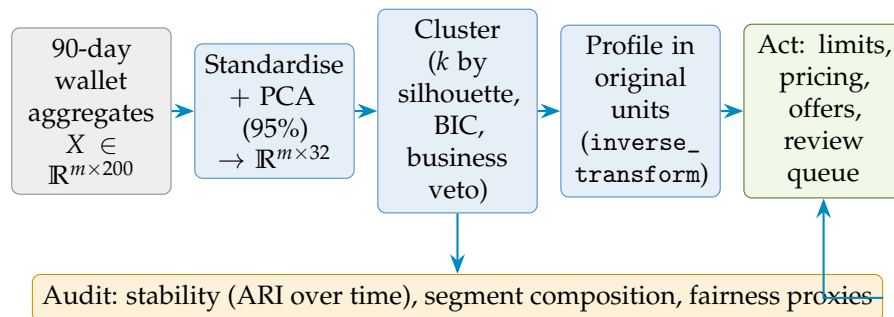
Kernel PCA ([Schölkopf et al., 1998](#)) deserves a word because it exposes a general difficulty. With no labels, there is no obvious criterion for choosing the kernel or its bandwidth  $\gamma$ . Two defensible options exist: select by downstream supervised performance, grid-searching the kernel jointly with the final estimator inside one pipeline; or select by lowest pre-image reconstruction error, which keeps the procedure fully unsupervised. Choosing by eye, because one embedding “looks more separated”, is not a method.

**Table 5.4:** The Chapter 5 toolbox at a glance.

| Method           | Model class          | Key knobs             | Reach for it when...                       |
|------------------|----------------------|-----------------------|--------------------------------------------|
| K-means          | centroid, hard       | $k, n\_init$          | fast segmentation of convex groups         |
| DBSCAN / HDBSCAN | density              | $\epsilon, minPts$    | $k$ unknown, odd shapes, noise wanted      |
| Gaussian mixture | probabilistic        | $k$ , covariance type | soft membership, densities, anomaly scores |
| PCA              | linear projection    | $d$ or variance kept  | compression, decorrelation, speed          |
| Kernel PCA       | nonlinear projection | kernel, $\gamma$      | curved structure, moderate $m$             |

## 5.8 Case Study: The Segmentation Pipeline in Production

We can now assemble the answer to both product questions from the opening case study. The architecture is deliberately boring, which is the point: Fig. 5.5 is one scikit-learn Pipeline object that is fitted once, versioned, and served.



**Figure 5.5:** The production segmentation pipeline. Everything left of the “Act” box is a single fitted artefact; the audit branch is not optional overhead but the mechanism by which the segments remain defensible.

### 5.8.1 Profiling: turning clusters into personas

A cluster index is not a segment. To make it one, invert the transform chain and describe each centroid in units a product manager recognises: transactions per week, median value in KES, share of inflows arriving on a single day of the month, agent-withdrawal ratio. Table 5.5 shows the shape of the result on the case-study data.

**Table 5.5:** Segments recovered from 90-day wallet behaviour, described in original units. *Illustrative profiles:* regenerate from your own lab run before quoting them.

| Segment            | Signature behaviour                          | Cash-flow rhythm     | Lending implication                                     |
|--------------------|----------------------------------------------|----------------------|---------------------------------------------------------|
| Market traders     | many small P2P sends, frequent float top-ups | daily, high velocity | short-cycle working capital, small limits, fast renewal |
| Gig workers        | irregular inflows from platform pay-bills    | weekly, volatile     | flexible repayment dates keyed to inflow, not calendar  |
| Salaried users     | one large inflow near month-end              | monthly, stable      | standard instalments, larger limits                     |
| Agri-cycle earners | sparse activity with post-harvest spikes     | seasonal             | bullet repayment aligned to harvest                     |
| Dormant / thin     | very low activity, sporadic airtime only     | none discernible     | insufficient signal; do not lend on behaviour alone     |

The fifth segment is the most valuable finding and the one a purely statistical reading would miss. It is not a market to be priced; it is a population for which the behavioural feature set carries no usable signal, and the correct action is to decline to score rather than to score badly.

### 5.8.2 Stability: a segment that moves is not a segment

Segments drift with the economy, the product mix and the acquisition channel. Before shipping, measure stability explicitly: re-fit on adjacent time windows or bootstrap resamples and compute the adjusted Rand index between partitions. An ARI near 1 indicates a reproducible structure; an ARI of 0.3 indicates that you have fitted noise and that next quarter's segments will not be this quarter's.

```
import joblib
from sklearn.metrics import adjusted_rand_score

# 1. Temporal stability: does the partition survive to the next window?
seg_q1 = build_segmenter(k=5).fit(X_q1)
seg_q2 = build_segmenter(k=5).fit(X_q2)
common = X_q1.index.intersection(X_q2.index)
ari = adjusted_rand_score(seg_q1.predict(X_q1.loc[common]),
                          seg_q2.predict(X_q2.loc[common]))
print(f"quarter-over-quarter ARI = {ari:.3f}") # < 0.5 -> do not ship

# 2. Composition audit: are segments proxying a protected attribute?
audit = (meta.loc[common]
         .assign(segment=seg_q1.predict(X_q1.loc[common]))
         .groupby("segment")[["gender", "county"]]
         .value_counts(normalize=True))
```

```
# 3. Ship one artefact: scaler + PCA + clusterer travel together.
joblib.dump(seg_q1, "artifacts/segmenter_v1.joblib")
```

**Listing 5.5:** Stability audit and serving artefact. The same fitted pipeline that produced the segments is the object that scores new wallets.

Shipping the whole pipeline as one object, as in [Listing 5.5](#), eliminates train–serve skew by construction: there is no way for the serving path to standardise with different means or project onto different components, because it is literally the same fitted estimator. This is the artefact that gets wrapped in a FastAPI endpoint in [Chapter 10](#) and monitored for drift in [Chapter 11](#).

#### Segmentation under Kenyan law

Unsupervised does not mean unaccountable. Three obligations bind this pipeline.

**Automated profiling.** The Data Protection Act, 2019 governs the processing of personal data, including inferences derived from it. A behavioural segment attached to an identified wallet is personal data; lawful basis, purpose limitation and data minimisation apply to the derived segment exactly as to the raw transaction log, and a data subject’s rights of access and objection extend to it.

**Pricing and terms.** The Central Bank of Kenya’s Digital Credit Providers Regulations, 2022 require pricing to be disclosed and non-exploitative, and require that credit decisions be explicable. A segment used to set a limit or a rate must therefore be describable in plain language: “high-velocity trader with daily float turnover”, not “cluster 3 of the PCA space”. This is why the profiling step in [Fig. 5.5](#) is load-bearing rather than cosmetic.

**Proxy discrimination.** Clusters fitted on behaviour can silently reconstruct protected or sensitive attributes: county of residence, language of merchant paybills, and transaction timing all correlate with demographics. Differential pricing across such segments can be discriminatory in effect without any protected attribute appearing in the model. Audit segment composition against the attributes you hold (as in [Listing 5.5](#)), document the result, and be prepared to justify any material disparity, before the regulator asks, not after.

**Chapter summary in one workflow**

Scale  $\rightarrow$  reduce (PCA, 95%)  $\rightarrow$  cluster ( $K$ -means, DBSCAN or GMM, with  $k$  chosen by silhouette or BIC and vetoed by the business)  $\rightarrow$  profile the result in original units  $\rightarrow$  audit stability and composition  $\rightarrow$  visualise with  $t$ -SNE or UMAP as a sanity check  $\rightarrow$  ship the whole pipeline as one versioned artefact. Anomaly detection reuses the same machinery: either the mixture's log-density or the PCA reconstruction error, thresholded at review capacity.

**Lab 5: Customer Segmentation and Anomaly Detection (1.5 h)**

**Notebook:** lab05\_unsupervised.ipynb.

**Data:** simulated mobile-money wallet aggregates,  $m \approx 150,000$  wallets  $\times n = 187$  behavioural features, unlabelled, with mixed scales, correlated feature blocks and injected anomalies.

1. **Reduce.** Build a Pipeline of StandardScaler  $\rightarrow$  PCA( $n\_components=0.95$ ). Plot cumulative explained variance and report  $d^*$ . Inspect the top three components' loadings and name them.
2. **Cluster.** Sweep  $K$ -means over  $k \in [2, 12]$ ; record inertia and silhouette. Produce a silhouette diagram at your chosen  $k$ .
3. **Compare.** Fit DBSCAN with  $\varepsilon$  from the  $k$ -distance knee. Report the number of clusters and the noise rate, and compare its partition to  $K$ -means with the adjusted Rand index.
4. **Model densities.** Fit Gaussian mixtures over  $k$  and three covariance types; select by BIC, rejecting any model with a component of effective count below 50. Flag the least likely 0.5% by `score_samples` and inspect those wallets.
5. **Visualise.** Use PCA 2D for visualizing clusters
6. **Profile and audit.** Name each segment, give one lending-relevant insight per segment, compute quarter-over-quarter ARI, and identify one plausible fairness concern with the evidence for it.

*Deliverable:* the executed notebook plus a segment-profile table, committed to your course repository before the next session.

## Exercises

1. **(Inertia monotonicity.)** Prove that the optimal value of Eq. (5.1) is non-increasing in  $k$ , and give the value at  $k = m$ . Explain in two sentences why this makes “choose  $k$  to minimise inertia” vacuous, and why the elbow heuristic is not.
2. **(Silhouette by hand.)** Consider the one-dimensional dataset  $\{0, 1, 2, 10, 11, 12\}$  with the partition  $C_1 = \{0, 1, 2\}$ ,  $C_2 = \{10, 11, 12\}$ . Compute  $s(i)$  for each point and the score  $\bar{s}$ . Now compute  $\bar{s}$  for the partition  $C'_1 = \{0, 1\}$ ,  $C'_2 = \{2, 10, 11, 12\}$  and comment on what the comparison shows.
3. **(Scaling changes the answer.)** Construct a two-dimensional dataset of six points, and a pair of units for the two axes, such that  $K$ -means with  $k = 2$  returns one partition on the raw data and a different partition after standardisation. State which partition you would defend to a product owner, and why.
4. **(DBSCAN boundaries.)** Give a concrete dataset,  $\varepsilon$  and  $\text{minPts}$  for which a border point is density-reachable from two distinct clusters, and show that its final label depends on visitation order. Then explain why Proposition 5.3 nonetheless makes DBSCAN reproducible for practical purposes.
5. **(EM for a two-component mixture.)** For  $k = 2$  in one dimension with known equal variances  $\sigma^2$  and equal weights, write  $\gamma_{i1}$  explicitly as a logistic function of  $x_i$ , and identify its slope and its midpoint in terms of  $\mu_1, \mu_2$  and  $\sigma^2$ . Comment on what happens to the slope as  $\sigma^2 \rightarrow 0$  and relate your answer to Proposition 5.5.
6. **(Degeneracy.)** Show that the likelihood of a one-dimensional two-component Gaussian mixture is unbounded above by exhibiting a parameter sequence along which  $\ell \rightarrow \infty$ . Explain precisely how  $\text{reg\_covar} > 0$  removes the divergence, and what price is paid for it.
7. **(Parameter counting and BIC.)** Derive the number of free parameters  $p$  for a  $k$ -component Gaussian mixture in  $\mathbb{R}^n$  under each of full, tied, diag and spherical. Evaluate  $p$  for  $n = 32$ ,  $k = 5$ , and determine how much larger  $\ln \hat{L}$  must be for full to beat diag on BIC with  $m = 150,000$ .
8. **(Concentration, numerically.)** Simulate  $m = 1000$  points with i.i.d.  $\text{Uniform}(0, 1)$  coordinates for  $n \in \{2, 10, 100, 1000\}$  and report  $(D_{\max} - D_{\min})/D_{\min}$  for a random query point in each case. Then

repeat with coordinates generated from a 5-dimensional latent factor model, and explain the difference with reference to Proposition 5.6.

9. **(PCA equivalence, applied.)** Using Proposition 5.7, prove that the mean squared reconstruction error of a  $d$ -component PCA equals  $\sum_{t>d} \lambda_t$ . For the covariance matrix of Example 5.2, verify this numerically for  $d = 1$  by explicitly projecting and reconstructing three points of your choice.
10. **(Governance.)** Your segmentation produces a cluster whose members receive a limit 40% lower than the population average, and whose composition is 78% from two counties against a 31% population base rate. Set out, in no more than a page: what you would measure next; which of the two Kenyan instruments discussed in this chapter are engaged; two remedies with their costs; and the circumstances under which shipping the segment unchanged would nonetheless be defensible.

## Key Terms

**Inertia** The within-cluster sum of squared distances to centroids, Eq. (5.1); the  $K$ -means objective. Monotone in  $k$ .

**Lloyd's algorithm** The alternating assign-and-update scheme that finds a local minimum of the inertia (Proposition 5.1).

**$k$ -means++** Distance-proportional seeding with an  $\mathcal{O}(\log k)$  expected approximation guarantee.

**Silhouette coefficient** Per-instance measure of cluster fit in  $[-1, 1]$ , Eq. (5.3); averaged into the silhouette score.

**Core / border / noise point** The DBSCAN trichotomy induced by  $\varepsilon$  and  $\text{minPts}$  (Definition 5.4).

**Density-reachability** The transitive relation over core points whose connected components are DBSCAN clusters.

**Responsibility**  $\gamma_{ij} = p(z_i = j \mid x_i)$ , the posterior probability that component  $j$  generated observation  $i$ .

**Expectation–Maximisation** Alternating maximisation of the ELBO in  $q$  and  $\theta$ ; never decreases the log-likelihood (Proposition 5.4).

**BIC / AIC** Complexity-penalised likelihood criteria, Eq. (5.9), used to select the number of mixture components.

**Curse of dimensionality** The joint effect of exponential sampling requirements and distance concentration (Proposition 5.6).

**Principal component** An eigenvector of the sample covariance; the top  $d$  span the variance-maximising, equivalently reconstruction-error-minimising, subspace.

**Explained variance ratio**  $\lambda_r / \sum_t \lambda_t$ ; its cumulative curve selects  $d$ .

**Reconstruction error**  $\|x - WW^\top x\|^2$ ; equals  $\sum_{t>d} \lambda_t$  on average, and doubles as an anomaly score.

**Manifold learning** Nonlinear dimensionality reduction assuming the data lies near a curved low-dimensional sheet.

**Perplexity**  $t$ -SNE's effective neighbourhood size, set by binary search on the per-point bandwidth  $\tau_i$ .

**Adjusted Rand index** Chance-corrected agreement between two partitions; the standard stability measure for a shipped segmentation.

## Further Reading

Géron (2022, Ch. 8–9) is the primary companion reading and mirrors this chapter's code idioms; Bishop and Bishop (2024, mixture models and EM) supplies the probabilistic depth behind Section 5.5. For the primary sources: Lloyd (1982) and Arthur and Vassilvitskii (2007) on  $K$ -means and its seeding; Ester et al. (1996) and Campello et al. (2013) on density-based clustering; Dempster et al. (1977) on EM; Rousseeuw (1987) on silhouettes; Pearson (1901); Hotelling (1933) on the origins of PCA, with Schölkopf et al. (1998) for its kernel extension and Halko et al. (2011) for the randomised algorithms that make it practical at scale. On the geometry of high dimension, Beyer et al. (1999) is the standard reference. For embeddings, read van der Maaten and Hinton (2008) and McInnes et al. (2018) for the methods and Wattenberg et al. (2016) before interpreting any map you produce. Huyen (2022, Ch. 6) connects the stability and monitoring concerns of Section 5.8 to the production material of Chapters 10 and 11.

**Part II**

**Deep Learning and Generative  
AI**



**Part III**

**MLOps and Production  
Systems**



# Appendices



# Capstone Project

---



# Solutions to Exercises

---

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument — framing before formalism, business units before metrics, honesty caveats stated explicitly — that the assignments and the capstone are graded against.

## Chapter 1: Introduction to Machine Learning

**Solution to Exercise 1 (Framing).** (a) Streaming fraud detection with a 300-alerts/day operations team.

*Mitchell specification.*

- *T*: score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- *E*: historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- *P*: fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@*k* with  $k = 300/\text{day}$ , weighted by transaction value.

*Framing block.* **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction

scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of [Section 1.4](#).

**(b) Thirty-day readmission from clinical pathway data.**

*Mitchell specification.*

- *T*: at discharge, estimate the probability that the patient is readmitted within 30 days.
- *E*: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

*Framing block.* **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

**Solution to Exercise 2 (ML or not).**

**(a) Blocking transactions above a regulatory limit: rule.** The requirement is a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

**(b) SIM-swap account takeover: ML, wrapped in rules.** The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion.

The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within  $N$  hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

**(c) Routing support tickets by language: ML, but off the shelf.** Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

### Solution to Exercise 3 (ERM: mean and median as optimal constants).

**Squared error.** For a constant predictor  $c$ , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting  $J'(c) = 0$  gives  $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$ , the sample mean;  $J''(c) = 2 > 0$  confirms the minimum.

**Absolute error.**  $G(c) = \frac{1}{n} \sum_i |y_i - c|$  is convex and piecewise linear, so we argue by subgradient. For  $c$  not equal to any  $y_i$ ,

$$G'(c) = \frac{1}{n} \left( \#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below  $c$  and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median (for even  $n$ , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

**Connection to [Section 1.6.2](#).** Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between [Eq. \(1.4\)](#) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

### Solution to Exercise 4 (Cost-sensitive thresholds).

**Two-way derivation.** For an applicant with calibrated default probability  $\hat{p}$ , compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when  $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$ ; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is Eq. (1.6). The derivation *uses* calibration:  $\hat{p}$  must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

**Three-way extension.** Add the action *refer*, costing a review fee  $c_R$  and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are  $\hat{p} c_{FN}$  (approve),  $(1 - \hat{p}) c_{FP}$  (decline), and  $c_R$  (refer). Comparing pairwise:

- approve beats refer iff  $\hat{p} c_{FN} < c_R$ , i.e.  $\hat{p} < t_L := c_R / c_{FN}$ ;
- decline beats refer iff  $(1 - \hat{p}) c_{FP} < c_R$ , i.e.  $\hat{p} > t_U := 1 - c_R / c_{FP}$ .

The optimal policy is therefore: *approve* if  $\hat{p} < t_L$ , *refer* if  $t_L \leq \hat{p} \leq t_U$ , *decline* if  $\hat{p} > t_U$ . The referral band is non-empty iff  $t_L < t_U$ , i.e.

$$c_R < \frac{c_{FN} c_{FP}}{c_{FN} + c_{FP}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold  $t^*$  — which, one checks, always lies inside  $[t_L, t_U]$  when the band exists. This is exactly the “ $\hat{p}$  near  $t^*$  routes to a human” edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

#### Solution to Exercise 5 (Leakage audit).

**(a) Number of repayment reminders sent: target leakage.** Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

**(b) Median mobile-money balance over the 90 days before application: safe — conditional on discipline.** The window  $[t_0 - 90d, t_0)$  respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at  $t_0$  (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

**(c) Standard deviation of balance over the loan term: temporal leakage.** The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

**(d) Merchant-ID one-hot with random row-wise splits: group leakage.** Repeat merchants appear on both sides of a random split, so the identifier

lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of  $t_0$ ), which are legitimate.

### Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in Section 1.9.1:  $y = g(x) + \varepsilon$  with  $\mathbb{E}[\varepsilon] = 0$ ,  $\text{Var}(\varepsilon) = \sigma^2$ ,  $\varepsilon$  independent of the training sample  $S$ ;  $\hat{f}$  is trained on  $S$ ;  $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$ . Fix  $x$  and expand (all expectations over  $S$  and  $\varepsilon$  jointly):

**First cross-term.** Expanding  $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$ , the middle term factorizes because  $\varepsilon$  is independent of  $S$  and  $\hat{f}$  is a function of  $S$  (and  $g(x)$  is a constant for fixed  $x$ ):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of  $\varepsilon$  kills the product. This leaves  $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$ .

**Second cross-term.** Insert and subtract  $\bar{f}$ :  $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$ , so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here  $(g - \bar{f})$  is a deterministic quantity (both terms are fixed numbers once  $x$  is fixed), so it exits the expectation; and by the *definition* of  $\bar{f}$ ,  $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$ . The cross-term vanishes, leaving  $\text{Bias}^2 + \text{Variance} + \sigma^2$ , which is Eq. (1.7). Summary of where each assumption bites: independence of  $\varepsilon$  and  $S$  kills the first cross-term; the definition of  $\bar{f}$  as the  $S$ -average kills the second.

### Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(8401)
n, d = 30_000, 10
t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
X = rng.normal(size=(n, d))
w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
```

```

drift = 2.5 * t                                     # concept drifts over time
logits = X @ w + drift * X[:, 3] - 3.2
y = rng.binomial(1, 1 / (1 + np.exp(-logits)))

model = Pipeline([("s", StandardScaler()),
                  ("m", LogisticRegression(max_iter=2_000,
                                           class_weight="balanced"))])

# (a) Random shuffled split -- time ignored
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=8401)
pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[: , 1]

# (b) Temporal split -- train on the past, test on the future
cut = int(0.75 * n)                                # rows already time-ordered
pb = (model.fit(X[:cut], y[:cut])
      .predict_proba(X[cut:]))[: , 1]

print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

**Listing B.1:** Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

**Direction and explanation.** The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

### Solution to Exercise 8 (Fairness).

Let  $F_A, F_B$  be the score CDFs of the calibrated model  $\hat{p}$  in the two regions, with base rates  $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$  and  $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$  (calibration plus the tower property force the group mean score to equal the group base rate). Approval is  $\hat{p} \leq t$  for a single global  $t$ .

**Impossibility across thresholds.** Demographic parity at *every* threshold would mean  $F_A(t) = F_B(t)$  for all  $t$ , i.e. identical score distributions — hence identical means, contradicting  $0.03 \neq 0.09$ . So there is no global-threshold policy family satisfying parity uniformly.

**The single-threshold case.** At one fixed  $t$ , parity requires the accident  $F_A(t) = F_B(t)$  at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case,  $F_B$  is stochastically dominated by — i.e. sits below —  $F_A$ ), giving  $F_B(t) < F_A(t)$ : region B's approval rate is strictly lower at every useful threshold. Even in contrived

cases where the CDFs cross at some  $t$ , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in [Section 1.11](#).

**Proposal to the regulator.** A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through [Eq. \(1.6\)](#)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as [Section 1.11](#) prescribes.

### [Solution to Exercise 9 \(Hybrid architecture\).](#)

A full-marks answer reproduces the topology of [Fig. 1.9](#) with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline ([Listing 1.5](#)), served behind the versioned API of [Listing 1.7](#). It alone emits  $\hat{p}$ ; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.
- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with  $\hat{p} \in [t_L, t_U]$  — the referral band derived in Solution 4 — route to a credit officer, with the agent’s brief attached.

**Where calibration must be preserved end-to-end** — the marking crux: the probability  $\hat{p}$  must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The

agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of [Eq. \(1.6\)](#), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer's *inputs*, never at its output.

## Chapter 2: Data Preparation and Feature Engineering

**Solution to Exercise 2.1.** Consider the electronic medical records of a network of Kenyan outpatient clinics.

**MCAR.** A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients' rows are affected is determined by the timing of the outage, not by the patients or their values:  $P(M | X) = P(M)$  ([Definition 2.1](#)). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

**MAR.** Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself ([Definition 2.2](#)). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

**MNAR.** HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient's own (unobserved) status and risk perception:  $P(M | X)$  depends on  $X_{\text{mis}}$  ([Definition 2.3](#)). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per [Remark 2.1](#), the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

**Solution to Exercise 2.2.** Let  $R_i \in \{0, 1\}$  indicate that row  $i$  is a complete case, and let  $X_i$  be its (complete-data) value. MCAR states  $P(R | X) = P(R)$ ,

i.e.  $R \perp X$ . The density of  $X$  among complete cases is, by Bayes' rule,

$$f(x | R = 1) = \frac{P(R = 1 | X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random sub-sample. Consequently, with  $n_{cc} = \sum_i R_i$  the (random) number of complete cases,

$$\mathbb{E} \left[ \frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \mid n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern  $(R_1, \dots, R_n)$  — under which each retained  $X_i$  has mean  $\mathbb{E}[X]$  by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size.  $\square$

**Solution to Exercise 2.3.** With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction  $\hat{\mathbb{E}}[X_{\text{mis}} | X_{\text{obs}}]$  of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X | X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X | X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column's sample variance therefore converges to  $\text{Var}(\mathbb{E}[X | X_{\text{obs}}])$ , understating  $\text{Var}(X)$  by the entire residual component  $\mathbb{E}[\text{Var}(X | X_{\text{obs}})]$ , and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a  $t$ -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and  $p$ -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create  $M$  completed datasets, and combine estimates with Rubin's rules, which add the between-imputation variance the point-estimate shortcut discards.

**Solution to Exercise 2.4.** Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming  $|z| > 3$  rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in

**Section 2.3).** Deleting  $|z| > 3$  rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training–serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of **Section 2.6**. A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian  $|z| > 3$  rule flags a large fraction of perfectly legitimate rows (the z-score’s  $\mu$  and  $\sigma$  are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

**Solution to Exercise 2.5.** From Eq. (2.7), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let  $m$  be the median. Its scaled value is  $m' = (m - x_{\min}) / (x_{\max} - x_{\min})$ . Replace the maximum by  $10 x_{\max}$  (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10 x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10 x_{\max} - x_{\min}}.$$

For  $x_{\min} = 0$  this ratio is exactly  $1/10$ : the median’s scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of **Eq. (2.8)**.

**Solution to Exercise 2.6.** Within level  $c$ , model each target as conditionally Bernoulli,  $y_i \mid \theta_c \sim \text{Bernoulli}(\theta_c)$ , and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m \bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is  $\alpha / (\alpha + \beta) = \bar{y}$  (the global rate) and the prior strength — the number of pseudo-observations — is  $\alpha + \beta = m$ . Observing  $n_c$  rows of level  $c$  with  $s_c = \sum_i y_i = n_c \bar{y}_c$  successes, conjugacy gives the posterior  $\theta_c \mid \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$ , whose mean is

$$\mathbb{E}[\theta_c \mid \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m \bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is [Eq. \(2.11\)](#) exactly. The limits confirm the shrinkage behaviour: as  $n_c \rightarrow 0$ ,  $\text{TE}(c) \rightarrow \bar{y}$  (an unseen or rare level is pulled entirely to the prior), and as  $n_c \rightarrow \infty$ ,  $\text{TE}(c) \rightarrow \bar{y}_c$  (abundant evidence overwhelms the prior). The smoothing parameter  $m$  is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label ([Section 2.5.3](#)).  $\square$

**Solution to Exercise 2.7.** *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which is the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of [Section 2.7.2](#), hiding behind two innocuous-looking columns that pass a univariate correlation screen.

*A screen for pairwise leakage.* Extend the univariate screen of [Section 2.7](#) to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by  $|\rho(\cdot, y)|$  or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model’s. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

**Solution to Exercise 2.8.** Assume the hash of [Eq. \(2.10\)](#) assigns each of the  $K$  levels independently and uniformly to one of  $d$  buckets. Two given levels collide with probability  $1/d$ , and there are  $\binom{K}{2}$  pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\text{\#collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For  $K = 10^4$  and  $d = 256$ :

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of  $K/d \approx 39$  levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off:

at this cardinality,  $d = 256$  buys a fixed 256-column footprint at the price of heavy aliasing, and raising  $d$  to  $2^{12}$ – $2^{14}$  reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a  $10^4$ -column one-hot encoding.

**Solution to Exercise 2.9.** *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp  $t$ , the feature ingests post- $t$  information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

*Redesign.* Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp,  $[t - 7d, t)$ , enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

**Solution to Exercise 2.10.** A five-sentence rebuttal. First, native NaN routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (Chapter 6), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training-serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage

hunting — is not imputation at all, and no amount of native NaN support dedupes a customer with three SIM cards.

## Chapter 3: Classical Machine Learning and Rigorous Evaluation

**Solution to Exercise 3.1.** A defensible committee-minute sentence: “*Holding every other feature in the model fixed, applicants with late-night transaction activity have  $e^{0.62} \approx 1.86$  times the odds of default of otherwise-identical applicants without it.*” The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association*, not a *causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the “effect” of late-night activity — can change substantially. It licenses no intervention claim (“discouraging late-night transactions would reduce defaults” does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio’s 3% base rate the two are numerically close ( $OR \approx RR$  when the outcome is rare), but the identification should still be avoided in a document regulators may read.

**Solution to Exercise 3.2.** *Derivative of the sigmoid.* With  $\sigma(z) = (1 + e^{-z})^{-1}$ ,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

*Gradient of the log loss.* Write the per-example loss with  $\hat{p} = \sigma(z)$ ,  $z = \theta^\top \mathbf{x}$ :

$$\ell(\theta) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\frac{y}{\hat{p}} \hat{p}(1 - \hat{p}) + \frac{1 - y}{1 - \hat{p}} \hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y) \hat{p} = \hat{p} - y,$$

and since  $\partial z / \partial \theta = \mathbf{x}$ ,  $\nabla_{\theta} \ell = (\hat{p} - y) \mathbf{x}$ . Averaging over the  $m$  training examples gives Eq. (3.4):  $\nabla_{\theta} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$ . The remarkable cancellation of the  $\hat{p}(1 - \hat{p})$  factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in Chapter 6.

**Solution to Exercise 3.3.** From Eq. (3.11),  $F_{\beta} = (1 + \beta^2)PR / (\beta^2 P + R)$ .

*Limit  $\beta \rightarrow 0$ :* the expression tends to  $PR/R = P$  directly.

Limit  $\beta \rightarrow \infty$ : divide numerator and denominator by  $\beta^2$ :

$$F_\beta = \frac{(1 + \beta^{-2}) PR}{P + \beta^{-2} R} \xrightarrow{\beta \rightarrow \infty} \frac{PR}{P} = R.$$

So  $\beta$  interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

*Numerical part.* With  $P = 0.40$ ,  $R = 0.75$ ,  $\beta = 2$ :

$$F_2 = \frac{(1 + 4) \cdot 0.40 \cdot 0.75}{4 \cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above  $F_1 = 2 \cdot 0.3/1.15 \approx 0.522$  because  $F_2$  rewards this classifier's recall-heavy profile.

**Solution to Exercise 3.4.** Fix an instance with calibrated probability  $p$  of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p c_{TP} + (1 - p) c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p c_{FN} + (1 - p) c_{TN}.$$

Flagging is optimal iff  $p c_{TP} + (1 - p) c_{FP} \leq p c_{FN} + (1 - p) c_{TN}$ , i.e.

$$(1 - p) (c_{FP} - c_{TN}) \leq p (c_{FN} - c_{TP}).$$

Under the “reasonableness” conditions  $c_{FP} > c_{TN}$  and  $c_{FN} > c_{TP}$  (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* = \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting  $c_{TP} = c_{TN} = 0$  recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves  $t^*$  unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ( $c_{TP} = c_{FP, \text{new}} - c_{FP, \text{old}} = 200$  added to the flag column), then with the case-study numbers  $t^* = (1000)/(1000 + 9800) \approx 0.093$ : review costs push the threshold up, and the scorer should flag slightly less eagerly.

**Solution to Exercise 3.5.** Let the positive scores be  $s_1^+, \dots, s_{m_+}^+$  and the negative scores  $s_1^-, \dots, s_{m_-}^-$ , and assume first that all scores are distinct. Sort all  $m_+ + m_-$  instances by decreasing score and trace the empirical ROC curve from  $(0, 0)$ : each positive encountered moves the curve *up* by  $1/m_+$ , each negative moves it *right* by  $1/m_-$ .

Consider the horizontal step contributed by a particular negative  $j$ . Its height above the FPR axis at that moment is

$$(\text{number of positives already passed})/m_+ = \#\{i : s_i^+ > s_j^-\}/m_+.$$

The area of that step is therefore  $\#\{i : s_i^+ > s_j^-\} / (m_+ m_-)$ . Summing over all negatives,

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{1}[s_i^+ > s_j^-] = \frac{U}{m_+ m_-},$$

where  $U = \sum_{i,j} \mathbb{1}[s_i^+ > s_j^-]$  is precisely the Mann–Whitney  $U$  statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability  $P(s^+ > s^-)$  for a uniformly random (positive, negative) pair, which is [Proposition 3.2](#).

*Ties.* If  $k_+$  positives and  $k_-$  negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at  $\frac{1}{2}$  each; correspondingly  $U$  is defined with  $\frac{1}{2}$  per tied pair. This yields the tie-corrected statement  $\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$  of [Eq. \(3.15\)](#).

**Solution to Exercise 3.6.** Suppose scores are independent of labels. Fix any threshold  $t$  and let  $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$  be the alert set. By independence, the label distribution *inside*  $A$  equals the marginal label distribution, so

$$\mathbb{E}[\text{Precision}(t)] = P(y = 1 \mid \mathbf{x} \in A) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR “curve” of a label-independent scorer is therefore the horizontal line at height  $\pi$ , and its area (average precision) is

$$\text{AP} = \int_0^1 \pi d(\text{recall}) = \pi.$$

On the case-study portfolio  $\pi \approx 0.03$ , so chance PR–AUC  $\approx 0.03$ . Reporting “PR–AUC = 0.31” without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of [Section 3.4.3](#) — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

**Solution to Exercise 3.7.** From [Eq. \(3.17\)](#),  $\text{se} = \sqrt{r(1-r)/n_+} \leq 0.02$  requires

$$n_+ \geq \frac{r(1-r)}{\text{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means  $525/0.03 = 17,500$  validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of [Section 3.6](#): on a small credit portfolio (say 10,000 labelled loans in total,  $\sim 300$  defaulters), *no single split can estimate recall to within two points*. The honest options

are exactly the chapter’s toolkit: aggregate information across stratified folds (Definition 3.7), attach bootstrap intervals to whatever held-out estimate is reported (Section 3.6.4), and refuse to interpret differences smaller than the resulting uncertainty band.

**Solution to Exercise 3.8.** With  $n_{01} = 63$  (only model B correct) and  $n_{10} = 41$  (only model A correct), Eq. (3.19) gives

$$\chi^2 = \frac{(|63 - 41| - 1)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since  $4.24 > \chi^2_{1,0.95} = 3.84$ , we reject at the 5% level the null hypothesis that the two models have equal error rates: model B’s advantage on the discordant cases is unlikely to be chance ( $p \approx 0.04$ ).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models’ accuracies by the same amount. McNemar’s test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative (Section 3.3.1).

**Solution to Exercise 3.9.** Applying Eq. (3.20) coordinate-wise with  $\lambda = 0.4$ :

$$(1, 0, 0) + 0.4 [(0, 1, 0) - (1, 0, 0)] = (1, 0, 0) + 0.4 (-1, 1, 0) = (0.6, 0.4, 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant’s region — and  $(0.6, 0.4, 0)$  asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder’s sum-to-one, single-one structure, and any coefficient interpretation (Example 3.1) built on “the region-A indicator” silently breaks.

*SMOTE-NC’s repair.* SMOTE-NC interpolates only the *numeric* coordinates via Eq. (3.20), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the  $k$  minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

*Residual risk.* The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as Section 3.7.2 already warned for plain

SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate (Section 3.5) before applying Proposition 3.1.

**Solution to Exercise 3.10.** Let  $w = c_{FN}/c_{FP}$  and suppose training re-weights each positive example by  $w$  (equivalently, imagine each positive duplicated  $w$  times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If  $p$  denotes the true (original-distribution) posterior  $P(y = 1 \mid \mathbf{x})$ , re-weighting the positive class by  $w$  transforms it to

$$p_w = \frac{w p}{w p + (1 - p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by  $w$ . Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \frac{1}{2} \iff w p \geq 1 - p \iff p \geq \frac{1}{1 + w} = \frac{c_{FP}}{c_{FP} + c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at  $t^*$  of Eq. (3.13). This is Elkan (2001)'s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

*Why try threshold moving first, then?* Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model's outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: “we score with model  $M$  and decline above 7.4% calibrated risk, derived from this cost matrix” is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link (Section 3.7.1).

## Chapter 4: Decision Trees, Ensembles and Hyperparameter Tuning

**Solution to Exercise 4.1.** *Independent voters.* With  $p = 0.55$ , the number of correct voters is  $K \sim \text{Binomial}(M, 0.55)$  and the majority is correct when

$K > M/2$ . The normal approximation  $K \approx \mathcal{N}(Mp, Mp(1-p))$  is justified for these  $M$  by the usual rule  $Mp(1-p) \gg 1$  (it equals 2.7, 25.0 and 247.7 respectively; for  $M = 11$  the approximation is rough and the exact sum is preferable, but it errs by under two points here). Writing  $z = (M/2 - Mp) / \sqrt{Mp(1-p)}$  and  $P_{\text{maj}} \approx 1 - \Phi(z) = \Phi(-z)$ :

| $M$  | $-z$  | $P_{\text{maj}}$ (normal) | $P_{\text{maj}}$ (exact) |
|------|-------|---------------------------|--------------------------|
| 11   | 0.332 | 0.630                     | 0.633                    |
| 101  | 1.005 | 0.843                     | 0.846                    |
| 1001 | 3.164 | 0.9992                    | 0.9992                   |

The committee converges on certainty, exactly as [Proposition 4.1](#) promises.

*Correlated blocks.* If the  $M$  voters consist of ten blocks of  $M/10$  perfectly correlated clones, every clone in a block casts the same vote, so the committee behaves as *ten* independent voters, whatever  $M$  is. For  $M = 1001$  ( $\approx 100$  clones per block) the relevant computation is the  $M = 10$  (even; take  $M = 11$  as the odd comparator) row, not the  $M = 1001$  row:  $P_{\text{maj}} \approx 0.63$ , and no increase of  $M$  at fixed block structure improves it. The *effective committee size* is the number of independent error patterns, not the number of members. This is [Eq. \(4.5\)](#) in disguise — within-block correlation  $\rho = 1$  contributes pure floor — and it is precisely the situation of a forest whose trees share their root split: trees that agree on the first partition of the data are partial clones, adding `n_estimators` buys little, and the productive intervention is decorrelation (`max_features`), not more trees.

**Solution to Exercise 4.2.** Let  $\bar{f} = \sum_m c_m f_m$  with  $\sum_m c_m = 1$ ,  $c_m \geq 0$ ,  $\text{Var}[f_m] = \sigma^2$  and pairwise correlation  $\rho$ . Then

$$\begin{aligned} \text{Var}[\bar{f}] &= \sum_m c_m^2 \sigma^2 + \sum_{m \neq m'} c_m c_{m'} \rho \sigma^2 \\ &= \sigma^2 \left[ (1 - \rho) \sum_m c_m^2 + \rho \left( \sum_m c_m \right)^2 \right] = \sigma^2 \left[ (1 - \rho) \sum_m c_m^2 + \rho \right], \end{aligned}$$

using  $\sum_{m \neq m'} c_m c_{m'} = (\sum_m c_m)^2 - \sum_m c_m^2 = 1 - \sum_m c_m^2$ . Since  $\rho \leq 1$ , minimising the variance means minimising  $\sum_m c_m^2$  subject to  $\sum_m c_m = 1$ . By Cauchy–Schwarz (or a Lagrange multiplier),  $\sum_m c_m^2 \geq \frac{1}{M} (\sum_m c_m)^2 = \frac{1}{M}$ , with equality iff  $c_m = 1/M$  for all  $m$ : *uniform weights are optimal*, and they recover [Proposition 4.2](#).

The optimality is conditional on the exchangeability assumptions. If the members have *unequal* variances — equivalently, unequal skill, as in [Listing 4.2](#) where a tuned LightGBM is simply a better estimator than the logistic baseline — the same Lagrangian calculation with member variances  $\sigma_m^2$  (and  $\rho = 0$  for clarity) gives optimal weights  $c_m \propto 1/\sigma_m^2$ : precision weighting. Soft voting with unequal weights is the practical, integer-granularity version of this, and stacking ([Section 4.8](#)) is its data-driven completion — the meta-learner *estimates* the weights instead of assuming exchangeability.

**Solution to Exercise 4.3.** *The limit without logarithms.* From  $1 - x \leq e^{-x}$  with  $x = \frac{1}{n}$ ,

$$\left(1 - \frac{1}{n}\right)^n \leq e^{-1}.$$

For the matching lower bound, apply  $1 - x \geq e^{-x/(1-x)}$  (valid for  $0 \leq x < 1$ , and itself a rearrangement of  $1 + u \leq e^u$  with  $u = x/(1-x)$ ) to get  $\left(1 - \frac{1}{n}\right)^n \geq e^{-n \cdot \frac{1/n}{1-1/n}} = e^{-n/(n-1)} \rightarrow e^{-1}$ . The two bounds pinch the sequence to  $e^{-1}$ . For  $n = 20$ ,  $\left(\frac{19}{20}\right)^{20} = 0.3585$ , already within 2.5% of  $e^{-1} \approx 0.3679$  — the asymptotics arrive early, which is why “37% out of bag” is a safe rule of thumb even for modest samples.

*Why an OOB-chosen threshold misbehaves.* (i) *Statistical:* each OOB prediction  $\bar{f}_{-i}^{\text{oob}}(x_i)$  in Eq. (4.6) averages only the  $\approx 0.37M$  trees that missed row  $i$ , whereas the deployed model averages all  $M$ . A mean of  $0.37M$  trees is a noisier, differently calibrated score than a mean of  $M$  trees, so the probability distribution on which the OOB threshold was optimised is not the distribution the production model emits; with the asymmetric costs of the case study (KES 10,000 vs KES 800), even a small calibration shift moves the cost-optimal cut-off materially. (ii) *Temporal:* the OOB estimate randomises over rows, so its implicit population is the historical mixture of the training window. The threshold’s economics depend on the *future* default base rate and score distribution, which drift; the temporal validation window of Chapter 3 is the only estimate that rehearses the deployment condition. In short, OOB may rank models acceptably, but the decision threshold should be set — and re-set, per Chapter 11 — on out-of-time data.

**Solution to Exercise 4.4.** After stage  $m$ , the new weight mass on the rows that  $f_m$  got wrong is

$$W_{\text{wrong}}^{(m+1)} = \sum_{i: f_m(x_i) \neq y_i} w_i^{(m)} e^{+\alpha_m} = e^{\alpha_m} \varepsilon_m W^{(m)}, \quad W_{\text{corr}}^{(m+1)} = e^{-\alpha_m} (1 - \varepsilon_m) W^{(m)},$$

where  $W^{(m)} = \sum_i w_i^{(m)}$ . Substituting  $e^{\alpha_m} = \sqrt{(1 - \varepsilon_m)/\varepsilon_m}$  from Proposition 4.4,

$$W_{\text{wrong}}^{(m+1)} = \sqrt{\frac{1 - \varepsilon_m}{\varepsilon_m}} \varepsilon_m W^{(m)} = \sqrt{\varepsilon_m (1 - \varepsilon_m)} W^{(m)} = W_{\text{corr}}^{(m+1)}.$$

The two masses are equal, so the weighted error of  $f_m$  on the updated weights is exactly  $\frac{1}{2}$ : each learner is rendered useless against the distribution it leaves behind. The implication is that AdaBoost can never profitably select the same hypothesis twice in a row — the re-weighting is an *orthogonalisation* step that forces every stage to contribute something the previous stage did not, which is the sequential counterpart of the decorrelation that bagging obtains by randomisation. (Hypotheses can recur later, once the weight distribution has moved on.)

**Solution to Exercise 4.5.** (a) *Binary log-loss.* With  $y \in \{0, 1\}$ , logits  $F$ , and  $\sigma(F) = (1 + e^{-F})^{-1}$ ,

$$L(y, F) = -[y \ln \sigma(F) + (1 - y) \ln(1 - \sigma(F))].$$

Using  $\sigma'(F) = \sigma(F)(1 - \sigma(F))$ ,

$$\frac{\partial L}{\partial F} = -\left[y \frac{\sigma'}{\sigma} - (1 - y) \frac{\sigma'}{1 - \sigma}\right] = -[y(1 - \sigma) - (1 - y)\sigma] = \sigma(F) - y,$$

so the pseudo-residual [Eq. \(4.7\)](#) is  $r = y - \sigma(F)$ : the error in predicted probability, as claimed in [Section 4.6.3](#).

(b) *Pinball loss.* At quantile level  $\tau \in (0, 1)$ ,

$$L_\tau(y, F) = \begin{cases} \tau(y - F), & y \geq F, \\ (1 - \tau)(F - y), & y < F, \end{cases} \quad r = -\frac{\partial L_\tau}{\partial F} = \begin{cases} \tau, & y > F, \\ \tau - 1, & y < F, \end{cases}$$

(the loss is not differentiable at  $y = F$ ; any subgradient value in  $[\tau - 1, \tau]$  serves). The boosted model then nudges its prediction up by a constant amount whenever it under-predicts and down whenever it over-predicts, with asymmetric magnitudes  $\tau$  and  $1 - \tau$ ; the fixed point of that asymmetry is the  $\tau$ -quantile.

*Use in collections.* A model of the  $\tau = 0.9$  quantile of repayment delay answers, per borrower, “by what day will this loan have been repaid with 90% confidence?” That is the planning quantity a collections operation needs: it sizes the follow-up queue conservatively, sets the day on which an unpaid loan becomes genuinely anomalous (rather than merely late-ish) and therefore worth an agent’s call, and prices expected recovery timelines into cash-flow forecasts. The conditional *mean* delay would understate all three, because delay distributions are heavy-tailed and the cost of under-provisioning is asymmetric — the same cost-sensitivity logic as [Chapter 3](#), applied to a regression target.

**Solution to Exercise 4.6.** Under squared error  $L = \frac{1}{2}(y - F)^2$  we have  $g_i = \partial_F L = F(x_i) - y_i = -r_i$  (minus the ordinary residual) and  $h_i = \partial_F^2 L = 1$ . Hence for leaf  $j$ ,  $G_j = -\sum_{i \in I_j} r_i$  and  $H_j = |I_j|$ , and [Eq. \(4.11\)](#) gives

$$w_j^* = -\frac{G_j}{H_j + \lambda} = \frac{\sum_{i \in I_j} r_i}{|I_j| + \lambda} \xrightarrow{\lambda=0} \frac{1}{|I_j|} \sum_{i \in I_j} r_i,$$

the mean residual in the leaf — so vanilla gradient boosting ([Section 4.6.3](#)) is recovered as the unregularised special case. For  $\lambda > 0$ ,

$$w_j^* = \frac{H_j}{H_j + \lambda} \cdot \underbrace{\frac{1}{H_j} \sum_{i \in I_j} r_i}_{\text{mean residual}},$$

a shrinkage of the mean residual by the factor  $H_j/(H_j + \lambda) \in (0, 1)$ . The factor is close to 1 for well-populated leaves and small for thin ones: a leaf holding  $H_j = 500$  loans with  $\lambda = 5$  keeps 99% of its estimate, while a leaf holding 5 loans keeps only 50%. The ridge penalty is thus an automatic scepticism dial — conclusions drawn from few borrowers are pulled toward “no adjustment” — which is exactly the behaviour one wants in a 4.1%-default portfolio where thin leaves are dominated by noise, and it complements the hard floor that `min_child_samples` imposes in [Listing 4.4](#).

**Solution to Exercise 4.7.** *Counterexample.* Hold every feature fixed except the delinquency feature `days_past_due` ( $x$ , in days), and suppose the unconstrained model’s fitted risk is piecewise constant:

$$F(x) = \begin{cases} 0.10, & 0 \leq x < 10, \\ 0.30, & 10 \leq x < 20, \\ 0.20, & 20 \leq x < 30, \\ 0.50, & x \geq 30. \end{cases}$$

Any reasonable notion of the *average* response is increasing — the least-squares linear fit of  $F$  on  $x$  has positive slope, and the overall trend runs from 0.10 to 0.50 — yet the model is not monotone: it assigns *lower* risk at  $x = 25$  than at  $x = 15$ .

*Governance failure.* Consider two applicants identical in every respect except that applicant A has 15 days of arrears and applicant B has 25. Suppose the approval threshold sits at 0.25: A is declined, B is approved. The adverse-action reason issued to A necessarily cites delinquency — it is the only feature in play — yet B, with *strictly worse* delinquency, was approved. The stated reason is therefore falsified by the lender’s own decisions: A’s appeal, a regulator’s file review, or a journalist’s paired test will surface the inconsistency, and under the automated-decision provisions of the Kenya DPA (2019) the lender must stand behind an explanation its model contradicts. The average trend is irrelevant to this failure because adverse-action reasons are given to *individuals*, who live at specific points of the curve, not on its regression line. The monotonic constraint of [Definition 4.5](#) removes the failure mode by construction, at the cost of the (noise-driven) dip — which is precisely the trade described in [Section 4.7.3](#).

**Solution to Exercise 4.8.** By [Proposition 4.5](#), an equal-resolution grid with  $B = 729$  and  $d = 6$  uses  $r = 729^{1/6} = 3$  distinct values per axis, so each of the two important hyperparameters is probed at just *three* values, each combination repeated across the  $3^4 = 81$  settings of the four irrelevant axes. Random search probes 729 distinct values of each important axis.

*Probability the grid never enters the best decile of both important axes simultaneously.* Model each axis as  $[0, 1]$  with the “best decile” an interval of length 0.1 whose location is unknown, and take the grid’s three values per axis

to be evenly spaced with a uniformly random offset,  $\{u, u + \frac{1}{3}, u + \frac{2}{3}\}$  with  $u \sim \text{U}[0, \frac{1}{3})$  (assumptions stated: independent axes, uniform target location, one-third spacing). Since the target interval  $0.1 < \frac{1}{3}$ , at most one grid value can fall in it, and the probability that one does is  $3 \times 0.1 = 0.3$  per axis. Because the grid is a Cartesian product, hitting the decile marginally on both important axes guarantees a trial at the intersection; the two axes are independent, so

$$P(\text{grid hits both}) = 0.3 \times 0.3 = 0.09, \quad P(\text{grid never does}) = 0.91.$$

For random search, a single trial lands in both deciles with probability  $0.1 \times 0.1 = 0.01$ , so

$$P(\text{random never does}) = (1 - 0.01)^{729} = 0.99^{729} \approx e^{-7.29} \approx 7 \times 10^{-4}.$$

The grid misses the jointly-best region nine times out of ten; random search finds it essentially always. This is Fig. 4.11 with numbers attached, and it is the practical content of low effective dimensionality (Bergstra and Bengio, 2012).

**Solution to Exercise 4.9.** With  $N = 81$ ,  $b = 25$  and  $\eta = 3$ , the rungs are:

| Rung                 | 1    | 2    | 3    | 4    | 5    |
|----------------------|------|------|------|------|------|
| Configurations alive | 81   | 27   | 9    | 3    | 1    |
| Budget each (rounds) | 25   | 75   | 225  | 675  | 2025 |
| Cost of rung         | 2025 | 2025 | 2025 | 2025 | 2025 |

Each rung costs exactly  $81 \times 25 = 2025$  configuration-rounds — the  $1/\eta$  survival rate and the  $\times \eta$  budget growth cancel — so the total is  $5 \times 2025 = 10,125$  configuration-rounds. Running all 81 configurations to the full 2,025 rounds would cost  $81 \times 2025 = 164,025$ : successive halving explores the same 81 candidates for 6.2% of the cost, a  $16.2\times$  saving, at the price of trusting rung-1 rankings.

*Where the trust fails.* The schedule assumes that performance after 25 boosting rounds predicts performance after 2,025. The clearest counterexample in our own chapter is the learning rate (Eq. (4.8)): a configuration with  $\nu = 0.02$  is deliberately slow — at 25 rounds it has barely moved from the base rate and looks terrible next to a  $\nu = 0.3$  configuration — yet it is exactly the kind of run that attains the *lowest* final validation loss. Aggressive halving systematically executes the low- $\nu$  candidates at rung 1 and crowns a fast-starting, earlier-overfitting configuration. Mitigations: hedge across brackets (Hyperband, Li et al., 2017), delay pruning (`n_warmup_steps` in Listing 4.6), or normalise the comparison by learning-curve extrapolation rather than raw score.

**Solution to Exercise 4.10.** Each measured score is  $\hat{\mathcal{V}}_t = v + \varepsilon_t$  with  $\varepsilon_t = sZ_t$  and  $Z_t \sim \mathcal{N}(0, 1)$  i.i.d. The search reports

$$\max_t \hat{\mathcal{V}}_t = v + s \max_t Z_t \implies \mathbb{E}[\max_t \hat{\mathcal{V}}_t] = v + s \mathbb{E}[\max_{t \leq T} Z_t],$$

since  $v$  is common and the maximum commutes with the shift. All  $T$  configurations are *equally good* by construction, so the entire excess over  $v$  is optimism manufactured by selection. Using the standard extreme-value approximation  $\mathbb{E}[\max_{t \leq T} Z_t] \approx \sqrt{2 \ln T}$ : for  $T = 100$ ,  $\sqrt{2 \ln 100} = \sqrt{9.21} \approx 3.03$ , so with fold-level noise  $s = 0.004$  in PR-AUC the expected reported best is

$$v + 0.004 \times 3.03 \approx v + 0.012.$$

An HPO study of one hundred trials will thus claim roughly  $+0.012$  PR-AUC that is not there, even when tuning does nothing at all. Two readings matter for Lab 4. First, the magnitude is the same order as the optimism gap students typically measure between `study.best_value` and the sealed out-of-time score — part of that gap is drift, but a predictable slice of it is pure winner’s curse, and Table 4.7’s tuned-model row would read  $\approx 0.28$  rather than  $0.294$  if the inner-loop best were taken at face value. Second, the formula shows the levers: optimism grows only as  $\sqrt{\ln T}$  (slowly — more trials are not catastrophic) but linearly in  $s$ , so the cheapest honesty gain is reducing fold noise: more folds, repeated CV, or larger validation windows. And, of course, reporting from data the search never touched (Fig. 4.13) removes the curse entirely — which is the point of the chapter’s final pitfall box.

## Chapter 5: Unsupervised Learning and Dimensionality Reduction

**Solution to Exercise 5.1.** Let  $J^*(k)$  denote the optimal inertia with  $k$  clusters. Take an optimal solution for  $k$  with centroids  $\mu_1, \dots, \mu_k$  and assignment  $c$ . Build a feasible  $(k + 1)$ -cluster solution by choosing any point  $x_r$  that is not already alone in its cluster, adding the centroid  $\mu_{k+1} = x_r$ , and reassigning  $x_r$  to it. Every other point keeps its assignment and hence its contribution to Eq. (5.1); the contribution of  $x_r$  falls from  $\|x_r - \mu_{c(r)}\|^2 \geq 0$  to 0. Therefore  $J^*(k + 1) \leq J^*(k)$ . At  $k = m$  we may place one centroid on each point, giving  $J^*(m) = 0$ .

Consequently “minimise inertia over  $k$ ” always returns  $k = m$ , which is the degenerate clustering in which every wallet is its own segment: the criterion is vacuous because model complexity is unpenalised. The elbow heuristic is not vacuous because it does not minimise  $J$ ; it examines the *rate* at which  $J$  falls and looks for the point past which additional clusters

buy little. That is an informal appeal to a complexity penalty, which BIC (Eq. (5.9)) makes formal — and this is precisely why BIC has a genuine interior minimum where inertia does not.

**Solution to Exercise 5.2.** Partition  $C_1 = \{0, 1, 2\}$ ,  $C_2 = \{10, 11, 12\}$ . By symmetry it suffices to do  $C_1$ .

- $x = 0$ :  $a = \frac{1+2}{2} = 1.5$ ;  $b = \frac{10+11+12}{3} = 11$ ;  $s = (11 - 1.5)/11 = 0.8636$ .
- $x = 1$ :  $a = \frac{1+1}{2} = 1$ ;  $b = \frac{9+10+11}{3} = 10$ ;  $s = (10 - 1)/10 = 0.9000$ .
- $x = 2$ :  $a = \frac{2+1}{2} = 1.5$ ;  $b = \frac{8+9+10}{3} = 9$ ;  $s = (9 - 1.5)/9 = 0.8333$ .

$C_2$  mirrors these exactly, so  $\bar{s} = \frac{1}{3}(0.8636 + 0.9000 + 0.8333) = 0.8657$ .

Partition  $C'_1 = \{0, 1\}$ ,  $C'_2 = \{2, 10, 11, 12\}$ .

- $x = 0$ :  $a = 1$ ;  $b = \frac{2+10+11+12}{4} = 8.75$ ;  $s = (8.75 - 1)/8.75 = 0.8857$ .
- $x = 1$ :  $a = 1$ ;  $b = \frac{1+9+10+11}{4} = 7.75$ ;  $s = 0.8710$ .
- $x = 2$ :  $a = \frac{8+9+10}{3} = 9$ ;  $b = \frac{2+1}{2} = 1.5$ ;  $s = (1.5 - 9)/9 = -0.8333$ .
- $x = 10$ :  $a = \frac{8+1+2}{3} = 3.6\bar{6}$ ;  $b = \frac{10+9}{2} = 9.5$ ;  $s = (9.5 - 3.6\bar{6})/9.5 = 0.6140$ .
- $x = 11$ :  $a = \frac{9+1+1}{3} = 3.6\bar{6}$ ;  $b = 10.5$ ;  $s = 0.6508$ .
- $x = 12$ :  $a = \frac{10+2+1}{3} = 4.3\bar{3}$ ;  $b = 11.5$ ;  $s = 0.6232$ .

$\bar{s} = \frac{1}{6}(0.8857 + 0.8710 - 0.8333 + 0.6140 + 0.6508 + 0.6232) = 0.4686$ .

**Comment.** The natural partition scores far higher (0.866 versus 0.469), so the silhouette correctly prefers it. More instructive is the single strongly negative value,  $s(2) = -0.833$ : the point  $x = 2$  is on average much closer to  $C'_1$  than to the cluster it was assigned to. This is exactly what the silhouette *diagram* is for. An analyst looking only at the mean would see a mediocre 0.469; an analyst looking at the per-point profile sees immediately *which* point is misassigned and can fix it. Note also that  $\bar{s}$  for the bad partition is still positive, because four of six points are content — averages hide misassignment, profiles do not.

**Solution to Exercise 5.3.** Take the six points, with axis 1 in Kenyan shillings and axis 2 as a count of transactions:

|               | $P_1$ | $P_2$ | $P_3$ | $P_4$ | $P_5$ | $P_6$ |
|---------------|-------|-------|-------|-------|-------|-------|
| $x_1$ (KES)   | 1000  | 1100  | 1200  | 9000  | 9100  | 9200  |
| $x_2$ (count) | 2     | 40    | 78    | 2     | 40    | 78    |

**Raw data.** Squared distances are dominated by  $x_1$ , whose spread is  $\sim 8000$  against  $\sim 76$  for  $x_2$ .  $K$ -means with  $k = 2$  therefore splits on value:  $\{P_1, P_2, P_3\}$  versus  $\{P_4, P_5, P_6\}$ . (Check: the within-cluster sum of squares for this split is  $2 \cdot (20000 + 2 \cdot 1352.7) \approx 4.5 \times 10^4$ , whereas splitting on count gives clusters spanning 8000 in  $x_1$  and costs  $\sim 1.3 \times 10^8$ .)

**Standardised data.** After centring and dividing by the sample standard deviations ( $\sigma_1 \approx 4271$ ,  $\sigma_2 \approx 34.9$ ), both axes have unit spread and the configuration becomes a symmetric  $2 \times 3$  grid at  $x_1 \approx \pm 0.94$  and  $x_2 \approx \{-1.09, 0, 1.09\}$ . The two candidate splits now cost the same by symmetry along  $x_1$ , but the three-level  $x_2$  axis has the larger total spread ( $\pm 1.09$  against  $\pm 0.94$ ), so  $K$ -means prefers to cut along  $x_2$ , returning e.g.  $\{P_1, P_4\}$  (low count) against  $\{P_2, P_3, P_5, P_6\}$ . Perturb the numbers slightly to break the tie in whichever direction you like; the point is that the two answers are different and both are “correct” for their respective metrics.

**Which to defend.** The standardised one — but the honest defence is not that standardisation is a rule. It is that the raw-data partition answers a question nobody asked: “are these transactions large or small in KES?” is answerable with a single threshold and needs no clustering at all. If the business genuinely wants value tiers, use a threshold and say so. Clustering earns its keep only when several dimensions jointly matter, and that requires them to be commensurable. Where a monetary feature is heavy-tailed, prefer a quantile or log transform to plain standardisation, since a handful of whale wallets will otherwise dominate the standardised axis too.

**Solution to Exercise 5.4. Construction.** Work in  $\mathbb{R}^2$  with  $\varepsilon = 1.05$  and  $\text{minPts} = 8$ . Build two clusters, each a dense blob plus one outlying “antenna” point:

- Cluster  $A$ : ten points inside a ball of radius 0.05 centred at  $(-0.6, 0)$ , plus a single point  $p_A = (0, 0)$ .
- Cluster  $B$ : ten points inside a ball of radius 0.05 centred at  $(2.6, 0)$ , plus a single point  $p_B = (2, 0)$ .
- The candidate border point  $q = (1, 0)$ .

*Every blob point is core:* it sees the other nine blob points (mutual distance  $\leq 0.1$ ) and its antenna (distance  $0.6 \pm 0.05 \leq \varepsilon$ ), so  $|N_\varepsilon| = 11 \geq 8$ . *Each antenna is core:*  $p_A$  sees its blob (ten points at  $\approx 0.6$ ), itself, and  $q$  (distance exactly  $1 \leq \varepsilon$ ), so  $|N_\varepsilon(p_A)| = 12 \geq 8$ .  $q$  is *not* core: its  $\varepsilon$ -ball contains only  $q$ ,  $p_A$  and  $p_B$ , since both blobs sit at distance  $\approx 1.6 > \varepsilon$ ; thus  $|N_\varepsilon(q)| = 3 < 8$ . Finally  $A$  and  $B$  are distinct components, because  $d(p_A, p_B) = 2 > \varepsilon$  and  $q$  — the only object between them — is not core, so it cannot propagate reachability.

Therefore  $q$  is a border point directly density-reachable from  $p_A$  and from  $p_B$ , which belong to different clusters. DBSCAN assigns  $q$  to whichever cluster is expanded first: process  $p_A$  before  $p_B$  and  $q$  joins  $A$ ; reverse the visitation order and  $q$  joins  $B$ . Its label is genuinely order-dependent.

**Why this does not undermine reproducibility.** By Proposition 5.3, the core-point set and the partition of core points into clusters are order-invariant. The ambiguity therefore never changes the number of clusters, their shape, or the membership of any core point — only the label of a point sitting in a low-density bridge, whose  $\varepsilon$ -ball is by construction almost empty. Operationally: the segments you name and price are determined by their cores, and a point whose label flips is exactly one about which you should not be making a confident decision. If bit-for-bit determinism is required for audit, fix the input ordering and record it; or use HDBSCAN, which assigns such points by membership strength rather than by visitation order.

**Solution to Exercise 5.5.** With  $k = 2$ ,  $\pi_1 = \pi_2 = \frac{1}{2}$ , and shared variance  $\sigma^2$ , the Gaussian normalising constants cancel in Eq. (5.5):

$$\gamma_{i1} = \frac{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right) + \exp\left(-\frac{(x_i - \mu_2)^2}{2\sigma^2}\right)} = \frac{1}{1 + \exp\left(\frac{(x_i - \mu_1)^2 - (x_i - \mu_2)^2}{2\sigma^2}\right)}.$$

Expanding the difference of squares,  $(x - \mu_1)^2 - (x - \mu_2)^2 = -2x(\mu_1 - \mu_2) + \mu_1^2 - \mu_2^2 = -2(\mu_1 - \mu_2)\left(x - \frac{\mu_1 + \mu_2}{2}\right)$ . Hence

$$\gamma_{i1} = \sigma_{\text{logistic}}(w(x_i - x_0)), \quad w = \frac{\mu_1 - \mu_2}{\sigma^2}, \quad x_0 = \frac{\mu_1 + \mu_2}{2},$$

where  $\sigma_{\text{logistic}}(z) = (1 + e^{-z})^{-1}$ . So the responsibility is exactly a logistic function of  $x$  with **slope parameter**  $w = (\mu_1 - \mu_2)/\sigma^2$  and **midpoint** at the arithmetic mean of the two component means, where  $\gamma_{i1} = \frac{1}{2}$ .

As  $\sigma^2 \rightarrow 0^+$  with  $\mu_1 \neq \mu_2$  fixed,  $|w| \rightarrow \infty$ : the logistic curve steepens into a step at  $x_0$ , so  $\gamma_{i1} \rightarrow \mathbb{1}\{x_i \text{ nearer } \mu_1\}$  for every  $x_i \neq x_0$ . This is precisely Proposition 5.5 in one dimension: the midpoint  $x_0$  is the perpendicular bisector (here, a single point), and the soft assignment collapses to  $K$ -means' hard nearest-centroid rule. The general lesson is that  $\sigma^2$  acts as a temperature: large  $\sigma^2$  gives soft, hedged memberships; small  $\sigma^2$  gives confident hard ones. Reporting a segment membership of 0.51 versus 0.49 as though it were a fact is a failure to read this parameter.

**Solution to Exercise 5.6. Divergence.** Take  $m \geq 2$  distinct points and the family

$$\theta_\epsilon : \quad \pi_1 = \pi_2 = \frac{1}{2}, \quad \mu_1 = x_1, \quad \sigma_1^2 = \epsilon, \quad \mu_2 = \bar{x}, \quad \sigma_2^2 = s^2 > 0 \text{ fixed.}$$

The first component's density at  $x_1$  is  $\mathcal{N}(x_1 \mid x_1, \epsilon) = (2\pi\epsilon)^{-1/2}$ , so the contribution of observation 1 to the log-likelihood is at least  $\log(\frac{1}{2}(2\pi\epsilon)^{-1/2}) \rightarrow +\infty$  as  $\epsilon \rightarrow 0^+$ . Every other observation  $i \geq 2$  retains the second component's contribution, bounded below by  $\log(\frac{1}{2}\mathcal{N}(x_i \mid \bar{x}, s^2)) > -\infty$ , which does not depend on  $\epsilon$ . Hence  $\ell(\theta_\epsilon) \rightarrow +\infty$ : the likelihood is unbounded above and no global maximiser exists. Geometrically, the fitted model has spiked onto a single data point and explains nothing.

**Effect of reg\_covar.** Adding  $\lambda > 0$  to the diagonal replaces  $\sigma_j^2$  by  $\sigma_j^2 + \lambda$ , so the achievable density at a point is bounded by  $(2\pi\lambda)^{-1/2}$  and the per-observation log-likelihood by  $\log(\pi_{\max}(2\pi\lambda)^{-1/2}) < \infty$ . Summing over  $m$  observations bounds  $\ell$  above, restoring existence of a maximum on the (now compact-in-effect) parameter region, and the degenerate spikes are removed from the search space entirely.

**Price paid.** Three costs. (i) The estimator is biased: every fitted covariance is inflated by  $\lambda$ , so genuinely tight clusters are reported as looser than they are, and density-based anomaly scores are correspondingly flattened. (ii) The reported likelihood, and therefore BIC, is no longer that of the model you claim to be fitting, so BIC comparisons are only valid at a *fixed*  $\lambda$ . (iii)  $\lambda$  is a scale-dependent hyperparameter: the same value means something different on standardised and unstandardised features, which is one more reason to fix the scaling before the mixture. In practice keep  $\lambda$  at the smallest value that prevents collapse (the scikit-learn default  $10^{-6}$  on standardised data is usually right) and treat a fit that needs a large  $\lambda$  as evidence that  $k$  is too big.

**Solution to Exercise 5.7.** Free parameters split into weights, means and covariances. Weights contribute  $k - 1$  (they sum to one); means contribute  $kn$ . Covariances:

| covariance_type | Covariance parameters | Total $p$                           |
|-----------------|-----------------------|-------------------------------------|
| full            | $k \frac{n(n+1)}{2}$  | $(k - 1) + kn + k \frac{n(n+1)}{2}$ |
| tied            | $\frac{n(n+1)}{2}$    | $(k - 1) + kn + \frac{n(n+1)}{2}$   |
| diag            | $kn$                  | $(k - 1) + 2kn$                     |
| spherical       | $k$                   | $(k - 1) + kn + k$                  |

For  $n = 32, k = 5$ :  $k - 1 = 4$  and  $kn = 160$ .

- full:  $\frac{32 \cdot 33}{2} = 528$  per component, so  $p = 4 + 160 + 5(528) = \mathbf{2804}$ .
- tied:  $p = 4 + 160 + 528 = 692$ .
- diag:  $p = 4 + 2(160) = \mathbf{324}$ .
- spherical:  $p = 4 + 160 + 5 = 169$ .

With  $m = 150,000$ ,  $\ln m = \ln(1.5 \times 10^5) \approx 11.9184$ . For full to beat diag on BIC we need

$$p_{\text{full}} \ln m - 2 \ln \hat{L}_{\text{full}} < p_{\text{diag}} \ln m - 2 \ln \hat{L}_{\text{diag}},$$

i.e.

$$\begin{aligned} \ln \hat{L}_{\text{full}} - \ln \hat{L}_{\text{diag}} &> \frac{1}{2}(2804 - 324)(11.9184) \\ &= \frac{1}{2}(2480)(11.9184) \approx \mathbf{14,778}. \end{aligned}$$

Averaged over the sample this is only  $14,778/150,000 \approx 0.0985$  nats per observation — about a 10% relative improvement in average likelihood. So with 150,000 wallets, full covariances are a *cheap* upgrade: modest correlation structure within segments will pay for 2480 extra parameters comfortably. Repeat the arithmetic at  $m = 2000$  and the required gap becomes  $\approx 0.5$  nats per observation, at which point diag usually wins. The general moral is that the “right” covariance type is a function of sample size, not of taste.

**Solution to Exercise 5.8. Independent coordinates.** A short simulation (numpy default generator, seed 0;  $m = 1000$  points and one query point drawn from the same distribution) gives ratios of the order

| $n$                   | 2   | 10   | 100  | 1000 |
|-----------------------|-----|------|------|------|
| i.i.d. coordinates    | 121 | 3.04 | 0.44 | 0.12 |
| 5-factor latent model | 179 | 10.4 | 8.88 | 7.55 |

(means over 20 replications; the  $n = 2$  column is enormous and wildly variable because  $D_{\min}$  can be arbitrarily small in low dimension, so read the *trend* across columns, not the individual digits). In the i.i.d. row the contrast between nearest and furthest neighbour collapses roughly like  $n^{-1/2}$ , exactly as Proposition 5.6 predicts from  $\text{Var}[D_n^2]/(\mathbb{E}[D_n^2])^2 = \mathcal{O}(1/n)$ .

**Latent factor model.** Generate  $u_i \in \mathbb{R}^5$  i.i.d. standard normal and set  $x_i = Au_i + 0.05\eta_i$  with  $A \in \mathbb{R}^{n \times 5}$  fixed and  $\eta_i$  small isotropic noise (second row of the table). The ratio now stays around 8–10 from  $n = 10$  all the way to  $n = 1000$ : raising the ambient dimension by two orders of magnitude does essentially nothing, because the distances are governed by the five latent coordinates and the extra measured dimensions carry almost no independent variability.

**Explanation.** The proof of Proposition 5.6 needs the coordinate contributions  $\delta_t$  to be (near-)independent, so that  $\text{Var}[D_n^2]$  grows like  $n$  while  $\mathbb{E}[D_n^2]$  grows like  $n$ , forcing the relative variance to 0. Under the factor model the  $\delta_t$  are strongly dependent: the effective, or intrinsic, dimension is 5, not  $n$ , and it is the intrinsic dimension that governs concentration. This is the entire justification for the chapter’s strategy. We do not escape the curse by wishing it away; we escape it because real behavioural features are redundant, and PCA finds the small number of directions along

which the data actually varies. It is also why one should measure the explained-variance curve before concluding that a 200-feature problem is high-dimensional: it may have an intrinsic dimension of 20.

**Solution to Exercise 5.9. Proof.** From the proof of Proposition 5.7,

$$E(W) = \frac{1}{m} \sum_i \|x_i\|^2 - \text{tr}(W^\top S W).$$

For centred data  $\frac{1}{m} \sum_i \|x_i\|^2 = \text{tr}(S) = \sum_{t=1}^n \lambda_t$ . At the optimum  $W = W_d$  we have, by Theorem 5.1,  $\text{tr}(W_d^\top S W_d) = \sum_{t \leq d} \lambda_t$ . Hence

$$E(W_d) = \sum_{t=1}^n \lambda_t - \sum_{t \leq d} \lambda_t = \sum_{t > d} \lambda_t,$$

as claimed: the mean squared reconstruction error is exactly the variance you threw away.

**Numerical verification.** With  $S = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$  from Example 5.2,  $\lambda_1 = 6$ ,  $\lambda_2 = 2$ ,  $v_1 = \frac{1}{\sqrt{2}}(1, 1)^\top$ . For  $d = 1$  the formula predicts  $E = \lambda_2 = 2$ . Take three centred points consistent with  $S$ , for instance

$$x_1 = (3, 1)^\top, \quad x_2 = (-1, -3)^\top, \quad x_3 = (-2, 2)^\top.$$

Projection:  $z_i = v_1^\top x_i$  gives  $z_1 = 4/\sqrt{2} = 2\sqrt{2}$ ,  $z_2 = -4/\sqrt{2} = -2\sqrt{2}$ ,  $z_3 = 0$ . Reconstruction  $\hat{x}_i = z_i v_1$ :  $\hat{x}_1 = (2, 2)^\top$ ,  $\hat{x}_2 = (-2, -2)^\top$ ,  $\hat{x}_3 = (0, 0)^\top$ . Residuals:  $x_1 - \hat{x}_1 = (1, -1)^\top$  with squared norm 2;  $x_2 - \hat{x}_2 = (1, -1)^\top$ , squared norm 2;  $x_3 - \hat{x}_3 = (-2, 2)^\top$ , squared norm 8. Mean =  $(2 + 2 + 8)/3 = 4$ .

The mean over this particular triple is 4, not 2, because three points do not reproduce  $S$ : their own sample covariance is  $\begin{pmatrix} 14/3 & 2/3 \\ 2/3 & 14/3 \end{pmatrix}$ , whose second eigenvalue is  $14/3 - 2/3 = 4$  — and 4 is exactly what the formula predicts for *their* covariance. The identity holds sample-wise, and checking it this way is a useful reminder that  $\lambda_t$  are properties of the sample you actually projected, not of an assumed population.

**Solution to Exercise 5.10.** A defensible one-page answer covers four things.

**What to measure next.** (i) Whether the disparity is in the *clustering* or in the *limit rule*: compute segment membership rates by county, then, within segment, compare limits — if limits are equal within segment, the mechanism is composition, not pricing. (ii) The behavioural distance between this segment and its neighbours: if the silhouette for this cluster is weak, the segment may be an artefact and the disparity a modelling choice rather than a finding. (iii) Whether the underlying features are themselves proxies: regress county on the top principal components and see how predictable it

is; high predictability means the “behavioural” representation substantially encodes geography. (iv) Outcome data where available — if this segment’s historical default rate genuinely differs, the disparity may be justified; if it does not, the 40% limit reduction lacks a risk basis.

**Instruments engaged.** Both discussed in the chapter’s regulatory box. The Data Protection Act, 2019 governs the derived segment as personal data: lawful basis, purpose limitation, and the data subject’s rights of access and objection apply to an inferred behavioural profile as much as to the raw transaction log. The CBK Digital Credit Providers Regulations, 2022 require pricing and terms to be disclosed and non-exploitative and credit decisions to be explicable — an unexplainable “cluster 3” limit rule does not meet that standard, and a limit reduction concentrated on two counties invites scrutiny of whether the differentiation is risk-based or merely correlational.

**Two remedies with costs.** (i) *Re-fit excluding geography-loaded features* (agent-location density, merchant-mix features tied to regional paybills). Cost: some genuine predictive signal is lost, so expected credit losses rise; and the remedy is incomplete, since geography can be reconstructed from residual features. (ii) *Keep the segmentation but sever it from pricing*, using segments only for product design and communication while limits are set by the audited, monotone scorecard of [Chapter 4](#) with fairness constraints. Cost: loses the operational simplicity of segment-based pricing, and requires a second governance process for the scorecard. A third, cheaper option worth listing: cap the maximum limit differential across segments, which bounds the harm without discarding the model.

**When shipping unchanged is defensible.** If the disparity is *explained* by a legitimate, disclosed, risk-relevant behavioural difference that survives conditioning on geography — for example, the segment is defined by seasonal single-harvest income, the lower limit reflects a measured higher default rate for bullet-repayment products, the county concentration is a consequence of where that livelihood occurs rather than of county itself, and an applicant from either county with non-seasonal income lands in a different segment. The test is counterfactual: does an otherwise identical borrower from a different county receive the same limit? If yes, the segment is tracking livelihood, not location, and the position is arguable — provided the reasoning, the tests and the residual disparity are documented before launch, and monitored afterwards.

# Bibliography

---

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.
- Arthur, D. and Vassilvitskii, S. (2007). *k*-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035.
- Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24.
- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Beyer, K., Goldstein, J., Ramakrishnan, R., and Shaft, U. (1999). When is “nearest neighbor” meaningful? In *International Conference on Database Theory (ICDT)*, pages 217–235.
- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. In *The World Bank Economic Review*, volume 34.
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.
- Breiman, L. (2001a). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L. (2001b). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.

- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.
- Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pages 160–172.
- Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.
- Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.
- Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 3rd edition.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35.
- Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.
- Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.
- McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- Melis, G., Dyer, C., and Blunsom, P. (2018). On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*.

- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.
- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.
- Prince, S. J. D. (2024). *Understanding Deep Learning*. MIT Press.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319.
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems* 28.
- Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.
- van der Maaten, L. and Hinton, G. (2008). Visualizing data using *t*-SNE. *Journal of Machine Learning Research*, 9:2579–2605.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Wattenberg, M., Viégas, F., and Johnson, I. (2016). How to use *t*-SNE effectively. *Distill*.
- Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2):241–259.
- Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.